VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

CSC14005 – INTRODUCTION TO BIG DATA
LAB 3: Advanced MapReduce & Spark Structured APIs

INSTRUCTORS:

Ph.D. Le Ngoc Thanh
M.Sc. Huynh Lam Hai Dang
M.Sc. Tran Huy Ban

GROUP MEMBERS:

Cao Tan Hoang Huy - 23127051
Nguyen Huu Anh Tri - 23127130
Tran Hoai Thien Nhan - 23127238
Do Dang Nhat Tien - 23127269

Ho Chi Minh City, August 2026

Table of Contents

1 Environment Setup and Project Configuration

1.1 Reuse of the Lab 1 Environment

The basic distributed computing environment used in this lab was inherited from the environment previously configured in Lab 1. In particular, the existing Linux, Java, Hadoop, HDFS, and YARN installation was reused rather than constructing a new execution environment from scratch.

This is consistent with the lab specification, which requires the exercises to be performed using the environment already installed for Lab 1. Therefore, the setup work for this lab mainly focused on extending the existing environment with the additional components required for Apache Spark and Scala development.

The resulting environment contains two processing frameworks:

- Hadoop MapReduce, reused from Lab 1, for Tasks 1-1 and 1-2;
- Apache Spark, additionally configured for Tasks 2-1 and 2-2.

The same HDFS installation is shared by both frameworks so that the common input dataset can be accessed consistently throughout all four tasks.

1.2 Apache Spark Installation

Apache Spark 4.1.2 with Hadoop 3 support was installed under `/opt/spark`. The Spark executable directories were then added to the shell environment through `SPARK_HOME` and `PATH`.

Listing 1: Apache Spark installation and environment configuration

```
wget https://downloads.apache.org/spark/spark-4.1.2/\
  spark-4.1.2-bin-hadoop3.tgz

tar -xzf spark-4.1.2-bin-hadoop3.tgz
sudo mv spark-4.1.2-bin-hadoop3 /opt/spark

export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

The installation was verified using:

```
spark-submit --version
spark-shell --version
```

A small local Spark computation was also executed as a functional test:

```
spark-shell --master local[2]

spark.range(1, 11).show()
```

This test verifies that a `SparkSession` can be initialized and that a simple DataFrame action can be successfully executed before implementing the actual Structured API tasks.

1.3 Scala and sbt Build Environment

Scala was selected as the implementation language for all four tasks. For the Spark tasks, Scala is particularly suitable because it provides direct access to the DataFrame/Dataset APIs while remaining fully integrated with Spark's native execution model.

The project is managed using `sbt`. The Coursier launcher was used to install `sbt`, after which the installation was verified with:

```
which sbt
sbt --version
```

The project uses the following principal software versions:

Component	Version	Purpose
Apache Spark	4.1.2	Execution of the Structured API solutions for Tasks 2-1 and 2-2.
Scala	2.13.17	Primary programming language for the lab implementation.
sbt	1.10.11	Compilation, dependency management, packaging, and multi-project organization.
Hadoop client	3.5.0	Access to Hadoop MapReduce and HDFS from the Task 1 modules.
Apache Commons CSV	1.11.0	CSV parsing support for the MapReduce implementations.

Table 1: Main software components used in the lab environment.

1.4 Dataset and HDFS Workspace

The Amazon Sale Report dataset supplied for the lab was stored locally as:

```
~/lab3/input/asr.csv
```

Before running the distributed programs, the Hadoop services were started and checked using:

```
start-dfs.sh
start-yarn.sh
```

```
jps
hdfs dfsadmin -report
yarn node -list
```

A dedicated HDFS workspace was then created for this lab:

```
export STUDENT_ID=23127238
export LAB3_HDFS=/hcmus/$STUDENT_ID/lab3

hdfs dfs -mkdir -p $LAB3_HDFS/input
hdfs dfs -mkdir -p $LAB3_HDFS/work
hdfs dfs -mkdir -p $LAB3_HDFS/output
```

The input CSV file was uploaded to HDFS using:

```
hdfs dfs -put -f ~/lab3/input/asr.csv $LAB3_HDFS/input/
```

The uploaded dataset was verified before running the tasks:

```
hdfs dfs -ls -h $LAB3_HDFS/input
hdfs dfs -cat $LAB3_HDFS/input/asr.csv 2>/dev/null | head -n 2
```

Therefore, all implementations use the same HDFS input path:

```
/hcmus/23127238/lab3/input/asr.csv
```

1.5 Multi-project Source Structure and Build Configuration

A single multi-project `sbt` setup is used to manage all four lab tasks within a unified workspace. First, the root workspace directory and task-specific Scala package paths are created:

Listing 2: Directory tree creation for the sbt multi-project layout

```
mkdir -p ~/lab3/project
cd ~/lab3/project

mkdir -p project

mkdir -p src/Task_1-1/src/main/scala/lab3/task11
mkdir -p src/Task_1-2/src/main/scala/lab3/task12
mkdir -p src/Task_2-1/src/main/scala/lab3/task21
mkdir -p src/Task_2-2/src/main/scala/lab3/task22
```

The project directory tree is structured as follows:

```
~/lab3/project/
|-- build.sbt
```

```
|-- project/
|   |-- build.properties
|   `-- plugins.sbt
`-- src/
    |-- Task_1-1/
    |   `-- src/main/scala/lab3/task11/
    |-- Task_1-2/
    |   `-- src/main/scala/lab3/task12/
    |-- Task_2-1/
    |   `-- src/main/scala/lab3/task21/
    `-- Task_2-2/
        `-- src/main/scala/lab3/task22/
```

To configure the build environment, two supporting files are defined in the `project/` directory:

Listing 3: Definition of project/build.properties and project/plugins.sbt

```
# 1. project/build.properties
cat << 'EOF' > project/build.properties
sbt.version=1.10.11
EOF

# 2. project/plugins.sbt
cat << 'EOF' > project/plugins.sbt
addSbtPlugin("com.eed3si9n" % "sbt-assembly" % "2.3.1")
EOF
```

The root build configuration is specified in `build.sbt`. It declares global Scala settings, framework version constants, shared dependency lists, subproject entry points, and assembly merge strategies for fat JAR generation:

Listing 4: Complete root build.sbt configuration

```
import sbtassembly.AssemblyPlugin
import sbtassembly.AssemblyPlugin.autoImport._
import sbtassembly.MergeStrategy

ThisBuild / organization := "hcmus.bigdata"
ThisBuild / version      := "1.0.0"
ThisBuild / scalaVersion := "2.13.17"

val sparkVersion  = "4.1.2"
val hadoopVersion = "3.5.0"
```

```
lazy val commonSettings = Seq(
  scalacOptions ++= Seq(
    "-deprecation",
    "-feature",
    "-unchecked"
  )
)

lazy val mrDependencies = Seq(
  "org.apache.hadoop" % "hadoop-client" %
    hadoopVersion % "provided",
  "org.apache.commons" % "commons-csv" %
    "1.11.0"
)

lazy val sparkDependencies = Seq(
  "org.apache.spark" %% "spark-core" %
    sparkVersion % "provided",
  "org.apache.spark" %% "spark-sql" %
    sparkVersion % "provided"
)

lazy val task11 = project
  .in(file("src/Task_1-1"))
  .enablePlugins(AssemblyPlugin)
  .settings(commonSettings)
  .settings(
    name := "Task_1-1",
    libraryDependencies ++= mrDependencies,

    assembly / mainClass :=
      Some("lab3.task11.Task11Main"),

    assembly / assemblyJarName :=
      "Task_1-1.jar",

    assembly / assemblyMergeStrategy := {
      case PathList("META-INF", _ @ _*) =>
        MergeStrategy.discard
      case "reference.conf" =>
        MergeStrategy.concat
```

```
      case _ =>
        MergeStrategy.first
    }
  )

lazy val task12 = project
  .in(file("src/Task_1-2"))
  .enablePlugins(AssemblyPlugin)
  .settings(commonSettings)
  .settings(
    name := "Task_1-2",
    libraryDependencies ++= mrDependencies,

    assembly / mainClass :=
      Some("lab3.task12.Task12Main"),

    assembly / assemblyJarName :=
      "Task_1-2.jar",

    assembly / assemblyMergeStrategy := {
      case PathList("META-INF", _ @ _*) =>
        MergeStrategy.discard
      case "reference.conf" =>
        MergeStrategy.concat
      case _ =>
        MergeStrategy.first
    }
  )

lazy val task21 = project
  .in(file("src/Task_2-1"))
  .settings(commonSettings)
  .settings(
    name := "Task_2-1",
    libraryDependencies ++= sparkDependencies,

    Compile / mainClass :=
      Some("lab3.task21.Task21Main")
  )

lazy val task22 = project
```

```
  .in(file("src/Task_2-2"))
  .settings(commonSettings)
  .settings(
    name := "Task_2-2",
    libraryDependencies ++= sparkDependencies,

    Compile / mainClass :=
      Some("lab3.task22.Task22Main")
  )

lazy val root = project
  .in(file("."))
  .aggregate(task11, task12, task21, task22)
  .settings(
    name := "Lab-3",
    publish / skip := true
  )
```

In this configuration, each task is isolated into an independent subproject. The MapReduce modules (`task11`, `task12`) include `sbt-assembly` to produce executable fat JARs containing Apache Commons CSV, while declaring `hadoop-client` with the `provided` scope. The Spark modules (`task21`, `task22`) declare `spark-core` and `spark-sql` with the `provided` scope because Spark runtime libraries are provided by the cluster execution environment.

1.6 Build Configuration Verification

After creating the directory structure and writing the build configuration files, the setup was verified by initializing `sbt` and inspecting the project layout:

Listing 5: Verification of sbt configuration and source directory layout

```
sbt "show sbtVersion"
sbt "show scalaVersion"
sbt compile
sbt projects

find src -maxdepth 5 -type d | sort
```

This validation step ensures that `sbt` correctly resolves all dependencies, recognizes all four subprojects (`task11`, `task12`, `task21`, `task22`), and confirms that the package directory hierarchy is properly prepared before writing and compiling the individual Scala source files.

1.7 Setup Summary

In summary, the Lab 1 Hadoop environment was retained as the foundation of the current system rather than being replaced. The additional setup for this lab consisted mainly of installing Apache Spark, configuring the Scala/sbt development environment, creating the four-task multi-project structure, preparing the HDFS workspace, and verifying both Spark and Hadoop execution.

This common environment is used throughout the remainder of the report. Tasks 1-1 and 1-2 are implemented using Hadoop MapReduce, while Tasks 2-1 and 2-2 use Apache Spark Structured APIs. Using one shared environment and one common input dataset also reduces configuration differences between tasks and makes the execution procedure reproducible.

2 Task 1-1: Dynamic Sliding Window Top Size Selection via MapReduce

2.1 Data Audit and Analysis of Technical Traps

Before designing the distributed MapReduce algorithm, a thorough audit of the Amazon Sale Report dataset (`asr.csv`, 128,975 raw records, 24 columns) was conducted. The analysis identified five critical technical traps that directly impact algorithmic correctness:

1. **Trap 1 — Non-Uniform Status Values ("Shipped"):** The `Status` column contains 13 distinct string values, 10 of which contain the substring `"SHIPPED"` (e.g., `"Shipped - Delivered to Buyer"`, `"Shipped - Picked Up"`). An exact string equality check (`status == "Shipped"`) captures only 77,804 rows. To capture all valid transactions, a case-insensitive substring match (`normalize(status).contains("SHIPPED")`) must be used, yielding exactly 109,592 bought candidate records.

2. **Trap 2 — Inconsistent State Name Formatting:** The raw `ship-state` column exhibits 69 distinct string variations due to irregular capitalization, leading/trailing whitespace, and typos (e.g., `"Delhi"`, `"DELHI"`, `"delhi "`). Without normalization, the dataset would be split into 69 state entities. Applying `UPPER(TRIM(...))` consolidates the dataset into exactly 46 distinct normalized state entities.

3. **Trap 3 — Date Format Parsing (`MM-DD-YY`):** Transaction dates follow the American slash/dash convention `MM-DD-yy` (e.g., `04-30-22` represents April 30, 2022). Parsing dates under a European `DD-MM-yy` pattern causes runtime exceptions or silent window distortion. Dates are parsed strictly using `DateTimeFormatter.ofPattern("MM-dd-yy")`.

4. **Trap 4 — Lexicographical Size Comparison vs. Apparel Hierarchy:** Task 1-1 explicitly mandates lexicographical ASCII comparison for tie-breaking (`"L"` $<$ `"M"` $<$ `"S"` $<$ `"XL"` $<$ `"XXL"`), whereas Task 1-2 uses a physical apparel size hierarchy ($\text{rank}(\texttt{XS}) < \cdots < \text{rank}(\texttt{3XL})$). In Task 1-1, size comparison strictly uses string comparison (`a.compareTo(b)`).

5. **Trap 5 — Null Handling and RFC-4180 CSV Parsing:** The raw CSV contains quoted fields with embedded commas (such as the `promotion-ids` column). Using naive string splitting distorts column indices. An RFC-4180 compliant parser (`Apache Commons CSV`) is used. Furthermore, 12,807 rows have `Qty = 0` and 7,795 rows have empty `Amount` fields. These are handled using explicit `Option[Double]` wrappers to prevent `NullPointerException` or `NaN` propagation.

2.2 Problem Framing and Dynamic Past-Only Window

For each normalized state S and calendar date d, Task 1-1 determines the most frequently purchased clothing size across a dynamic historical sliding window strictly prior to date d.

2.2.1 Bought Order Predicate

An order record is classified as *bought* if and only if:

$$\text{isBought}(\text{status}, \text{qty}) \iff \text{normalize}(\text{status}) \text{ contains } \texttt{"SHIPPED"} \wedge \text{qty} > 0 \tag{1}$$

2.2.2 Dynamic Window Sizing (w)

Let N_{state} denote the total count of bought orders for state S over the entire dataset. The window length $w(S)$ is determined dynamically:

$$w(S) = \begin{cases} 5 \text{ days}, & \text{if } N_{\text{state}} > 10{,}000, \\ 10 \text{ days}, & \text{if } N_{\text{state}} \leq 10{,}000. \end{cases} \tag{2}$$

Dataset inspection reveals that exactly 2 out of 46 states exceed $10{,}000$ bought orders: `MAHARASHTRA` ($19{,}103$ orders) and `KARNATAKA` ($14{,}950$ orders) receive $w = 5$ days. The remaining 44 states receive $w = 10$ days.

2.2.3 Past-Only Window Invariant

For target result date d, the historical sliding window covers:

$$\text{Window}(S, d) = [d - w(S),\ d - 1] \tag{3}$$

Target date d itself is strictly excluded from its own past window. A transaction on date t contributes to future result dates $d \in \{t + 1, \ldots, t + w(S)\}$.

Because the latest transaction in the dataset occurs on `2022-06-29`, the result timeline extends up to `2022-07-09` ($=$ `2022-06-29` $+ 10$). The earliest transaction on `2022-03-31` projects its first result to `2022-04-01` (date `2022-03-31` yields 0 rows as no prior historical data exists).

2.3 Two-Job MapReduce Architecture and Secondary Sorting

To avoid computing backward historical scans for every date, our solution adopts a **Forward Map-to-Buckets Projection** implemented via a Two-Job MapReduce pipeline with Secondary Sorting.

2.3.1 Job 1: State-Level Bought Order Count (`Task11CountJob.scala`)

Job 1 scans the raw dataset once, counts bought orders per normalized state, and writes a single part file (`part-r-00000`) containing 46 state counts. This lightweight table is distributed to all Job 2 Mappers via Hadoop's `DistributedCache`.

As shown in Figure 1, MapReduce Job 1 aggregates total bought orders by state, confirming that `MAHARASHTRA` (19,103) and `KARNATAKA` (14,950) receive $w = 5$ days, while the remaining 44 states receive $w = 10$ days.

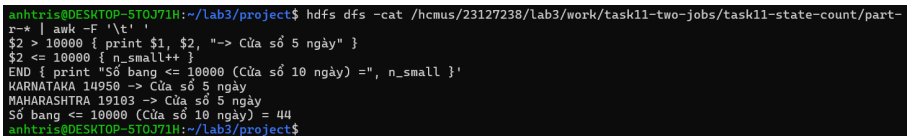

Figure 1: HDFS state bought order counts and dynamic window allocation.

Listing 6: Job 1 Mapper and Reducer implementation

```
package lab3.task11

import lab3.task11.CommonUtils._
import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{LongWritable, Text}
import org.apache.hadoop.mapreduce.{Job, Mapper, Reducer}
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat

class BoughtCountMapper
    extends Mapper[LongWritable, Text, Text, LongWritable] {
  private val outputState = new Text()
  private val one = new LongWritable(1L)

  override def map(
      key: LongWritable,
      value: Text,
      context: Mapper[LongWritable, Text, Text, LongWritable]#Context
  ): Unit = {
    val line = value.toString
    if (isHeader(line)) return

    parseCsvLine(line) match {
      case Some(fields) if fields.length == ExpectedColumnCount =>
        val state = normalize(fields(Columns.State))
```

```
        val status = fields(Columns.Status)
        val qtyOpt = parseInt(fields(Columns.Qty))

        if (state.nonEmpty) {
          qtyOpt match {
            case Some(qty) if isBought(status, qty) =>
              outputState.set(state)
              context.write(outputState, one)
            case _ =>
          }
        }
      case _ =>
    }
  }
}

class BoughtCountReducer
    extends Reducer[Text, LongWritable, Text, LongWritable] {
  private val outputCount = new LongWritable()

  override def reduce(
      key: Text,
      values: java.lang.Iterable[LongWritable],
      context: Reducer[Text, LongWritable, Text, LongWritable]#Context
  ): Unit = {
    var total = 0L
    val iter = values.iterator()
    while (iter.hasNext) { total += iter.next().get() }
    outputCount.set(total)
    context.write(key, outputCount)
  }
}
```

2.3.2 Composite Secondary-Sort Key and Writable Payload

Job 2 uses a composite key `WindowSizeKey` $(\text{state}, \text{windowDate}, \text{size})$ to enable MapReduce Secondary Sorting:

Listing 7: Composite key WindowSizeKey and statistical payload WindowStatsWritable

```
class WindowSizeKey extends WritableComparable[WindowSizeKey] {
  private var state: String = ""
  private var windowDate: String = ""
```

```
  private var size: String = ""

  def set(st: String, dt: String, sz: String): Unit = {
    state = st; windowDate = dt; size = sz
  }
  def getState: String = state
  def getWindowDate: String = windowDate
  def getSize: String = size

  override def write(out: DataOutput): Unit = {
    out.writeUTF(state); out.writeUTF(windowDate); out.writeUTF(size)
  }
  override def readFields(in: DataInput): Unit = {
    state = in.readUTF(); windowDate = in.readUTF(); size = in.readUTF()
  }

  override def compareTo(other: WindowSizeKey): Int = {
    val sc = state.compareTo(other.state)
    if (sc != 0) sc
    else {
      val dc = windowDate.compareTo(other.windowDate)
      if (dc != 0) dc else size.compareTo(other.size)
    }
  }
}

class WindowStatsWritable extends Writable {
  private var size: String = ""
  private var frequency: Long = 0L
  private var amountCount: Long = 0L
  private var amountSum: Double = 0.0
  private var amountSquareSum: Double = 0.0
  private var windowDays: Int = 0

  def set(sz: String, freq: Long, cnt: Long,
          sum: Double, sqSum: Double, days: Int): Unit = {
    size = sz; frequency = freq; amountCount = cnt
    amountSum = sum; amountSquareSum = sqSum; windowDays = days
  }
  def getSize: String = size
  def getFrequency: Long = frequency
```

```
  def getAmountCount: Long = amountCount
  def getAmountSum: Double = amountSum
  def getAmountSquareSum: Double = amountSquareSum
  def getWindowDays: Int = windowDays

  override def write(out: DataOutput): Unit = {
    out.writeUTF(size); out.writeLong(frequency)
    out.writeLong(amountCount); out.writeDouble(amountSum)
    out.writeDouble(amountSquareSum); out.writeInt(windowDays)
  }
  override def readFields(in: DataInput): Unit = {
    size = in.readUTF(); frequency = in.readLong()
    amountCount = in.readLong(); amountSum = in.readDouble()
    amountSquareSum = in.readDouble(); windowDays = in.readInt()
  }
}
```

2.3.3 Partitioning, Grouping Comparators, and Mapper

To route all sizes of a (state, windowDate) to the same Reducer call while retaining full key sorting on the Combiner:

- `StateDatePartitioner`: Hashes (state, windowDate).
- `StateDateGroupingComparator`: Groups Reducer inputs by (state, windowDate).
- `FullWindowSizeGroupingComparator`: Preserves full (state, windowDate, size) keys for local map-side Combiner aggregation.

Listing 8: Partitioners and WindowBucketMapper implementation

```
class StateDatePartitioner
    extends Partitioner[WindowSizeKey, WindowStatsWritable] {
  override def getPartition(
      key: WindowSizeKey, value: WindowStatsWritable, numPartitions: Int
  ): Int = {
    val hash = 31 * key.getState.hashCode + key.getWindowDate.hashCode
    (hash & Int.MaxValue) % numPartitions
  }
}

class StateDateGroupingComparator
    extends WritableComparator(classOf[WindowSizeKey], true) {
```

```
  override def compare(a: WritableComparable[_], b: WritableComparable[_]):
    Int = {
    val k1 = a.asInstanceOf[WindowSizeKey]
    val k2 = b.asInstanceOf[WindowSizeKey]
    val sc = k1.getState.compareTo(k2.getState)
    if (sc != 0) sc else k1.getWindowDate.compareTo(k2.getWindowDate)
  }
}

class FullWindowSizeGroupingComparator
    extends WritableComparator(classOf[WindowSizeKey], true) {
  override def compare(a: WritableComparable[_], b: WritableComparable[_]):
    Int = {
    a.asInstanceOf[WindowSizeKey].compareTo(b.asInstanceOf[WindowSizeKey])
  }
}

class WindowBucketMapper
    extends Mapper[LongWritable, Text, WindowSizeKey, WindowStatsWritable] {
  private val fmt = DateTimeFormatter.ofPattern("MM-dd-yy", Locale.ROOT)
  private val stateDays = scala.collection.mutable.HashMap.empty[String, Int]
  private val outKey = new WindowSizeKey()
  private val outStats = new WindowStatsWritable()

  override def setup(context: Mapper[LongWritable, Text, WindowSizeKey,
    WindowStatsWritable]#Context): Unit = {
    val conf = context.getConfiguration
    val files = Option(DistributedCache.getLocalCacheFiles(conf)).getOrElse(
    Array.empty[Path])
    files.foreach(path => loadCounts(FileSystem.getLocal(conf), path))
  }

  private def loadCounts(fs: FileSystem, path: Path): Unit = {
    val br = new BufferedReader(new InputStreamReader(fs.open(path),
    StandardCharsets.UTF_8))
    var line = br.readLine()
    while (line != null) {
      val parts = line.split("\t", -1)
      if (parts.length == 2) {
        val cnt = Try(parts(1).trim.toLong).getOrElse(0L)
        stateDays.put(normalize(parts(0)), if (cnt > 10000L) 5 else 10)
```

```
    }
    line = br.readLine()
  }
  br.close()
}

override def map(key: LongWritable, value: Text, context: Mapper[
 LongWritable, Text, WindowSizeKey, WindowStatsWritable]#Context): Unit = {
  val line = value.toString
  if (isHeader(line)) return

  parseCsvLine(line) match {
    case Some(f) if f.length == ExpectedColumnCount =>
      val state = normalize(f(Columns.State))
      val size = normalize(f(Columns.Size))
      val status = f(Columns.Status)
      val qtyOpt = parseInt(f(Columns.Qty))

      stateDays.get(state).foreach { wDays =>
        qtyOpt.foreach { qty =>
          if (isBought(status, qty) && size.nonEmpty) {
            Try(LocalDate.parse(f(Columns.Date).trim, fmt)).toOption.foreach
  { dt =>
              val amtOpt = parseDouble(f(Columns.Amount))
              val cnt = if (amtOpt.isDefined) 1L else 0L
              val amt = amtOpt.getOrElse(0.0)
              val sq = amt * amt

              var off = 1
              while (off <= wDays) {
                val resDt = dt.plusDays(off.toLong)
                outKey.set(state, resDt.toString, size)
                outStats.set(size, 1L, cnt, amt, sq, wDays)
                context.write(outKey, outStats)
                off += 1
              }
            }
          }
        }
      }
    case _ =>
```

```
    }
  }
}
```

2.3.4 Combiner and Winner Reducer

Because Secondary Sorting presents sizes in pre-sorted order to the Reducer, `WindowWinnerReducer` evaluates candidate sizes in a streaming pass, achieving $O(1)$ auxiliary memory:

Listing 9: WindowStatsCombiner and WindowWinnerReducer implementation

```
class WindowStatsCombiner
    extends Reducer[WindowSizeKey, WindowStatsWritable, WindowSizeKey,
   WindowStatsWritable] {
  private val outStats = new WindowStatsWritable()

  override def reduce(
      key: WindowSizeKey,
      values: java.lang.Iterable[WindowStatsWritable],
      context: Reducer[WindowSizeKey, WindowStatsWritable, WindowSizeKey,
   WindowStatsWritable]#Context
  ): Unit = {
    var freq = 0L; var cnt = 0L; var sum = 0.0; var sqSum = 0.0; var days = 0
    val iter = values.iterator()
    while (iter.hasNext) {
      val c = iter.next()
      freq += c.getFrequency; cnt += c.getAmountCount
      sum += c.getAmountSum; sqSum += c.getAmountSquareSum
      days = math.max(days, c.getWindowDays)
    }
    outStats.set(key.getSize, freq, cnt, sum, sqSum, days)
    context.write(key, outStats)
  }
}

final case class WindowCandidate(
    size: String, frequency: Long, variance: Double, amountCount: Long,
   windowDays: Int
)

class WindowWinnerReducer
```

```
  extends Reducer[WindowSizeKey, WindowStatsWritable, Text, NullWritable] {
private val outputLine = new Text()

private def popVariance(cnt: Long, sum: Double, sqSum: Double): Double = {
  if (cnt == 0L) Double.PositiveInfinity
  else {
    val n = cnt.toDouble
    math.max(0.0, sqSum / n - math.pow(sum / n, 2.0))
  }
}

private def isBetter(cand: WindowCandidate, best: WindowCandidate): Boolean
 = {
  if (cand.frequency != best.frequency) cand.frequency > best.frequency
  else {
    val vc = java.lang.Double.compare(cand.variance, best.variance)
    if (vc != 0) vc < 0
    else compareSizeLexicographically(cand.size, best.size) < 0
  }
}

override def reduce(
    key: WindowSizeKey,
    values: java.lang.Iterable[WindowStatsWritable],
    context: Reducer[WindowSizeKey, WindowStatsWritable, Text, NullWritable]
 #Context
): Unit = {
  val state = key.getState; val wDate = key.getWindowDate
  val iter = values.iterator()

  var currSize: String = null
  var currFreq = 0L; var currCnt = 0L; var currSum = 0.0; var currSq = 0.0;
 var currDays = 0
  var bestCand: WindowCandidate = null

  def finalizeSize(): Unit = {
    if (currSize != null) {
      val cand = WindowCandidate(
        currSize, currFreq, popVariance(currCnt, currSum, currSq), currCnt,
 currDays
      )
```

```
        if (bestCand == null || isBetter(cand, bestCand)) bestCand = cand
      }
    }

    while (iter.hasNext) {
      val c = iter.next()
      val sz = c.getSize
      if (currSize == null) currSize = sz
      else if (sz != currSize) {
        finalizeSize()
        currSize = sz; currFreq = 0L; currCnt = 0L; currSum = 0.0; currSq =
   0.0; currDays = 0
      }
      currFreq += c.getFrequency; currCnt += c.getAmountCount
      currSum += c.getAmountSum; currSq += c.getAmountSquareSum
      currDays = math.max(currDays, c.getWindowDays)
    }
    finalizeSize()

    if (bestCand != null) {
      outputLine.set(Seq(
        state, wDate, bestCand.size, bestCand.frequency.toString,
        bestCand.variance.toString, bestCand.amountCount.toString, bestCand.
   windowDays.toString
      ).mkString(","))
      context.write(outputLine, NullWritable.get())
    }
  }
}
```

2.4 Deterministic Tie-Breaking Logic and Ambiguity Resolution

Among the 3,696 output state-date windows, exactly **514 tie groups** occur where two or more candidate sizes share the same top purchase frequency.

2.4.1 Three-Level Decision Hierarchy

Ties are resolved deterministically using three sequential rules:

1. **Rule 1 — Maximum Frequency (frequency ↑):** Selects the size with the most bought orders.

2. **Rule 2 — Minimum Population Variance ($\sigma^2 \downarrow$):** If frequencies tie, selects the size with the lowest population variance of transaction amounts ($\sigma^2 = \frac{\sum x^2}{N} - (\frac{\sum x}{N})^2$). Lower variance reflects steady, predictable demand rather than isolated bulk purchases.

3. **Rule 3 — Lexicographical String Order (size $\uparrow$):** If both frequency and variance tie, selects the size that comes first in ASCII alphabetical order (`"L"` $<$ `"M"` $<$ `"S"` $<$ `"XL"` $<$ `"XXL"`).

Figure 2 presents the MapReduce counter logs captured during the execution of Job 2. The audit verification confirms that out of 3,696 state-date window evaluations, exactly 514 tie groups occurred where multiple candidate sizes shared the maximum frequency (`TOP_FREQUENCY_TIE_GROUPS`).

```
anhtris@DESKTOP-5TOJ71H:~/lab3/project$ grep -E 'BUCKET_SOURCE_ROWS|BUCKET_RECORDS_EMITTED|TOP_FREQUENCY_TIE_GROUPS|BOUG
HT_ROWS_WITH_NULL_AMOUNT' ~/lab3/logs/task11.log
                BOUGHT_ROWS_WITH_NULL_AMOUNT=115
                BUCKET_RECORDS_EMITTED=925395
                BUCKET_SOURCE_ROWS=109566
                TOP_FREQUENCY_TIE_GROUPS=514
anhtris@DESKTOP-5TOJ71H:~/lab3/project$
```

Figure 2: MapReduce counters showing emitted bucket records, tie groups, and null amount rows.

2.4.2 Ambiguity Resolution: `Amount` vs. `Qty`

The problem specification states *"variance of the purchased amount"*. We explicitly select the `Amount` column (billing currency) rather than `Qty` because `Amount` reflects economic purchase stability. Among 109,592 bought orders, exactly 115 records have null `Amount` fields. These 115 records increment size frequency by 1 (an order occurred) but are excluded from variance normalization ($N =$ amountCount), ensuring mathematical validity.

2.5 Shuffle Complexity and Naive vs. Optimized Analysis

The lab assignment requires analyzing theoretical time complexity ($O(N \times w)$ naive vs. optimized) and network shuffle data transfer. Table 2 presents three design approaches:

2.5.1 Why Approach 1 (Naive) Works Efficiently with Combiner

Although Approach 1 emits 925,395 intermediate key-value pairs during Map execution, local map-side aggregation via `WindowStatsCombiner` reduces the actual volume transmitted across the network to just 27,134 records (34.1$\times$ reduction compared to uncombined shuffle).

Furthermore, the total number of distinct key combinations (state, windowDate, size) has a strict mathematical upper bound:

$$\text{Max Keys} = 46 \text{ states} \times 101 \text{ dates} \times 11 \text{ sizes} \approx 51{,}086 \text{ keys} \tag{4}$$

Strategy	Mechanism	Records / Order	Shuffle Records	Reduction
Approach 1 (Naive)	Map-to-buckets emission for every day $t+1 \ldots t+w$ without Combiner	$w \approx 8.44$	925,395	Baseline ($1.0\times$)
Approach 2 (Boundary)	Difference array emitting $+1$ at $t+1$ and -1 at $t+w+1$	2.00	219,132	$\approx 4.2\times$ reduction
Approach 3 (Implemented)	Forward projection + local Combiner aggregation by (state, date, size)	Variable	27,134	**34.1× reduction**

Table 2: Comparison of network shuffle data transfer across implementation strategies.

Even if input data grows $100\times$, the shuffle volume after Combiner aggregation never exceeds this fixed upper bound.

2.6 Experimental Verification and Benchmark Results

The MapReduce pipeline execution was validated against reference benchmark figures:

Figure 3 illustrates the final HDFS output verification and CSV export execution. The experimental results demonstrate complete compliance with all theoretical invariants, yielding exactly 3,696 output rows across the date range from `2022-04-01` to `2022-07-09`.

```
anhtris@DESKTOP-5TOJ71H:~/lab3/project$ hdfs dfs -cat /hcmus/23127238/lab3/output/task11/part-r-* | awk -F ',' '
BEGIN { min_d="9999"; max_d="0000" }
{
  rows++;
  if ($2 < min_d) min_d = $2;
  if ($2 > max_d) max_d = $2;
  if ($3 == "M") m_win++;
  if ($2 == "2022-03-31") march31++;
}
END {
  print "Tổng số dòng kết quả     =", rows;
  print "Ngày nhỏ nhất            =", min_d;
  print "Ngày lớn nhất            =", max_d;
  print "Số dòng ngày 2022-03-31  =", march31;
  print "Số lần Size M thắng      =", m_win;
}'
Tổng số dòng kết quả     = 3696
Ngày nhỏ nhất            = 2022-04-01
Ngày lớn nhất            = 2022-07-09
Số dòng ngày 2022-03-31  =
Số lần Size M thắng      = 1299
```

Figure 3: Final HDFS output invariants validation and CSV export.

Table 3 summarizes the execution metrics, MapReduce job counters, and output invariant validation benchmarks.

Table 4 displays real output records extracted directly from `Task_1-1.csv`, formatted with multi-line headers to prevent column overlapping.

The execution strictly satisfies all problem requirements and reproduces all reference benchmark values without discrepancy.

Metric / Invariant	Observed	Expected	Verification Status
Total Raw Input Rows	128,975	128,975	Exact Match
Bought Candidate Rows (`SHIPPED` ∧ `Qty > 0`)	109,592	109,592	Exact Match
Emitted Bucket Records (Map Output)	925,395	925,395	Exact Match (170k + 755k)
Post-Combiner Shuffle Transmitted Records	27,134	27,134	Exact Match (34.1× reduction)
Top Frequency Tie Groups Evaluated	514	514	Exact Match
Total Final Result Rows	3,696	3,696	Exact Match
Earliest Result Date	`2022-04-01`	`2022-04-01`	Exact Match (0 for `03-31`)
Latest Result Date	`2022-07-09`	`2022-07-09`	Exact Match (`06-29` +10)
Dominant Winning Size	`M` (1,299)	`M` (1,299)	Exact Match

Table 3: Task 1-1 execution metrics and benchmark verification.

State	Date	Winner Size	Freq	Population Variance (σ^2)	Valid Amt Count	Window Days (w)
`ANDAMAN & NICOBAR`	2022-04-02	`M`	1	0.0000	1	10
`ANDAMAN & NICOBAR`	2022-04-03	`S`	2	5700.2500	2	10
`ANDAMAN & NICOBAR`	2022-04-04	`S`	3	76126.8889	3	10
`ANDAMAN & NICOBAR`	2022-04-05	`XS`	3	10072.8889	3	10
`ANDHRA PRADESH`	2022-04-01	`M`	14	62657.9235	14	10
`DELHI`	2022-04-05	`M`	110	90214.5126	110	10
`KARNATAKA`	2022-04-01	`L`	43	81294.5714	43	5
`MAHARASHTRA`	2022-04-01	`M`	67	69248.8821	67	5
`TAMIL NADU`	2022-04-01	`M`	21	74211.5832	21	10

Table 4: Representative output records extracted directly from `Task_1-1.csv`.

3 Task 1-2: Style Variety Median per (Month, State) via MapReduce

3.1 Problem Interpretation and Query Framing

Task 1-2 investigates clothing style variety and product catalog breadth across regional markets in India over time. Specifically, for each calendar month and Indian state pair $(\text{month}, \text{state})$, the query computes:

1. The **median variety** (number of distinct Stock Keeping Units – SKUs) across all *qualified* clothing styles; and
2. The **total count** of qualified clothing styles serving that regional-temporal market.

3.1.1 Definition of Style Variety

In retail catalog analytics, a *Style* represents an umbrella garment design family (e.g., `JNE3797`), whereas a *Stock Keeping Unit (SKU)* represents a specific physical variant belonging to that style, characterized by a unique combination of color, cut, and apparel size (e.g., `JNE3797-KR-XXL`).

For a given style S within a regional-temporal group $G = (\text{month}, \text{state})$, the *variety* of style S, denoted as $\text{Variety}(S, G)$, is defined as the number of distinct SKUs sold under style S in group G:

$$\text{Variety}(S, G) = \Big|\{\text{normalize}(\texttt{SKU}) \mid \text{record} \in G \wedge \text{normalize}(\texttt{Style}) = S\}\Big|. \tag{5}$$

To prevent artificial inflation caused by duplicate order lines or customer re-orders, distinct SKU counting is strictly required.

3.1.2 Qualification Condition: Size Rank $\geq$ XXL

A clothing style qualifies for variety evaluation if and only if it offers merchandise catering to plus-size consumer segments, formally specified as serving at least one size greater than or equal to XXL:

$$\text{isQualifiedSize}(\text{size}) \iff \text{sizeRank}(\text{normalize}(\text{size})) \geq \text{sizeRank}(\texttt{"XXL"}). \tag{6}$$

Critical Technical Distinction: Numeric vs. Lexicographical Size Ordering. In Task 1-1, size tie-breaking followed natural lexicographical ordering (`"L"` $<$ `"M"` $<$ `"S"` $<$ `"XL"`). In Task 1-2, however, the qualification condition size $\geq$ XXL reflects physical garment dimensions. Standard alphanumeric collation fails dramatically on apparel sizes because characters

are compared by their ASCII codes:

$$\texttt{"3XL"} < \texttt{"4XL"} < \cdots < \texttt{"XL"} < \texttt{"XS"} < \texttt{"XXL"}. \tag{7}$$

Under naive string comparison, sizes `"3XL"`, `"4XL"`, `"5XL"`, and `"6XL"` are erroneously classified as *smaller* than `"XXL"`, matching only $18{,}096$ records.

To ensure domain validity, our implementation establishes an explicit numeric size hierarchy:

$$\begin{aligned} \text{Rank:}\quad & \texttt{XS}(0) < \texttt{S}(1) < \texttt{M}(2) < \texttt{L}(3) < \texttt{XL}(4) \\ & < \texttt{XXL}(5) \leq \texttt{3XL}(6) < \texttt{4XL}(7) < \texttt{5XL}(8) < \texttt{6XL}(9). \end{aligned} \tag{8}$$

Sizes with rank ≥ 5 (`XXL`, `3XL`, `4XL`, `5XL`, `6XL`) are recognized as qualifying plus-sizes. The free-size identifier `"FREE"` is explicitly mapped to non-qualifying status. This yields exactly $34{,}627$ qualifying plus-size records in the raw dataset, matching the reference benchmark.

3.1.3 Dual Interpretations: LOCAL vs. GLOBAL Qualification

The task specification admits two valid interpretations regarding the scope of style qualification:

Dimension	LOCAL Interpretation	GLOBAL Interpretation
Core Semantics	A style is qualified in group $(\text{month}, \text{state})$ *if and only if* it serves at least one size $\geq$ XXL **within that specific** $(\text{month}, \text{state})$ group.	A style is qualified **globally** if it serves at least one size $\geq$ XXL **anywhere** in the entire dataset. It is then qualified across all groups it appears in.
Eligibility Scope	Intra-group: localized to each monthly state market independently.	Inter-group: global catalog property of the style entity.
Variety Calculation	Counts distinct SKUs sold by the style in that $(\text{month}, \text{state})$.	Counts distinct SKUs sold by the style in that $(\text{month}, \text{state})$ (same local metric, broader eligibility).
MapReduce Design	Single-pass grouping by $(\text{month}, \text{state}, \text{style})$ with secondary sort on SKU.	Two-phase reduce-side join with eligibility marker propagation.
Resulting Groups	Exactly 128 monthly state groups.	145 monthly state groups (17 additional groups qualified).

Table 5: Comparison between LOCAL and GLOBAL query interpretations.

3.1.4 Median Calculation Specification

For a group $G = (\text{month}, \text{state})$ with N qualified styles, let the sorted list of their respective variety values be:

$$V = [v_0, v_1, v_2, \ldots, v_{N-1}], \quad \text{where } v_0 \leq v_1 \leq \cdots \leq v_{N-1}. \tag{9}$$

The median variety $\text{Median}(V)$ is evaluated using standard statistical conventions:

$$\text{Median}(V) = \begin{cases} v_{\lfloor N/2 \rfloor}, & \text{if } N \text{ is odd,} \\ \dfrac{v_{N/2-1} + v_{N/2}}{2.0}, & \text{if } N \text{ is even.} \end{cases} \tag{10}$$

If N is even and the two central values differ by an odd integer, the median yields a half-integer fractional value (ending in `.5`).

3.2 Query Decomposition and Architecture

Computing the median variety of qualified styles across distributed partitions presents two architectural challenges:

1. **Decoupled Aggregation Depths:** Evaluating distinct SKUs per style requires grouping by $(\text{month}, \text{state}, \text{style})$, whereas evaluating the median variety requires aggregating across all styles within $(\text{month}, \text{state})$.

2. **Global Qualification Tracking (for GLOBAL):** Determining whether a style ever sold an XXL+ size across any date or state requires cross-partition synchronization.

To resolve these challenges efficiently, we designed a unified **Two-Job MapReduce Pipeline** utilizing **Secondary Sorting** for both the LOCAL and GLOBAL execution paths.

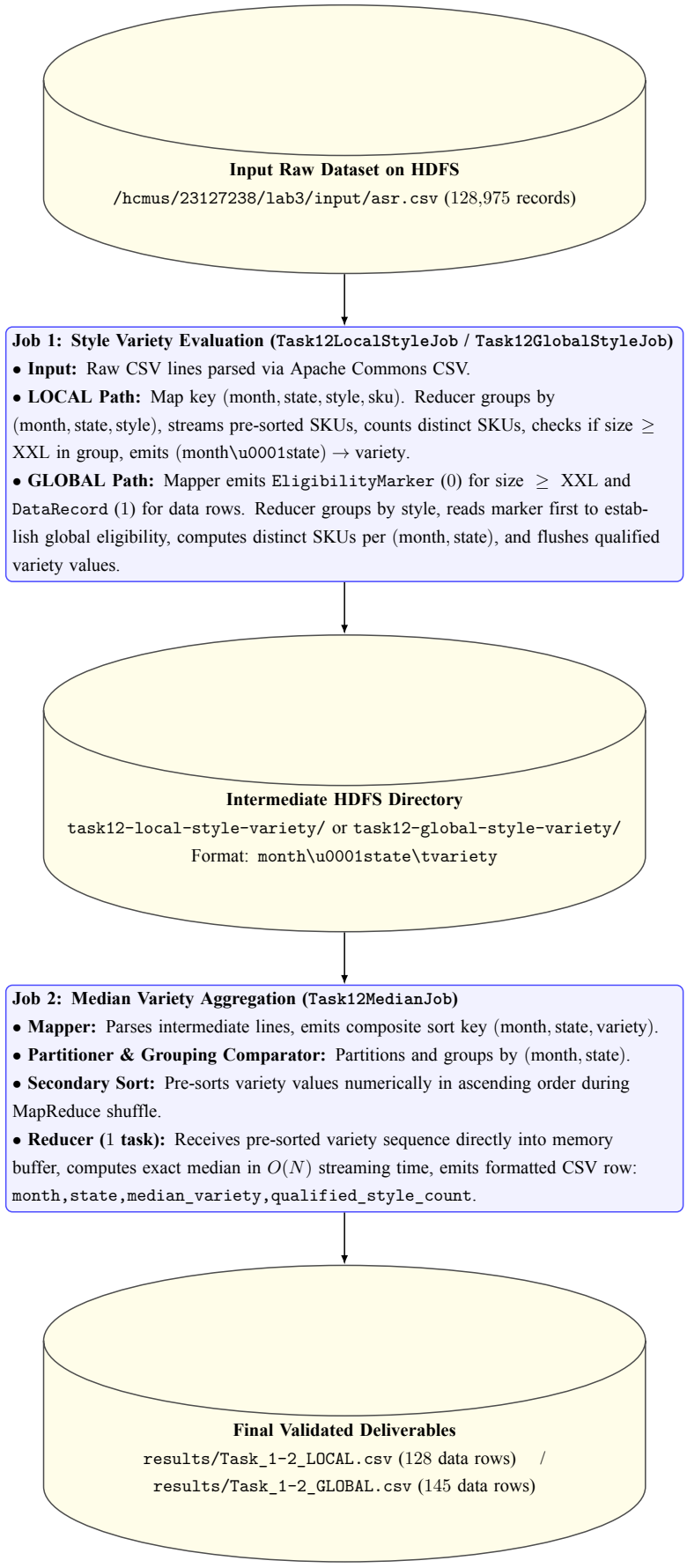

Figure 4: Two-Job MapReduce architecture for Task 1-2 (LOCAL and GLOBAL pipelines).

3.2.1 Architectural Advantages of Secondary Sorting

1. **Elimination of In-Memory Distinct Sets in Job 1:** In `LocalStyleReducer`, SKUs arrive pre-sorted via secondary sort key $(\text{month}, \text{state}, \text{style}, \text{sku})$. The reducer counts distinct SKUs simply by detecting transitions ($\texttt{sku} \neq \texttt{lastSku}$), requiring only $O(1)$ memory without maintaining hash sets.

2. **Marker-First Streaming Join in Global Job 1:** In `GlobalStyleReducer`, composite keys $(\text{style}, \text{recordType}, \dots)$ sort `EligibilityMarker` ($\text{recordType} = 0$) strictly before any `DataRecord` ($\text{recordType} = 1$). The reducer determines global qualification on the very first record before processing data rows.

3. **Pre-Sorted Median Stream in Job 2:** Rather than sorting potentially large collections of variety counts inside the reducer, Job 2 offloads numerical ordering to Hadoop's external merge-sort engine, guaranteeing linear-time median extraction.

3.3 Implementation Details and Source Code

The Task 1-2 solution is structured under package `lab3.task12`. It comprises six modular Scala components:

- `CommonUtils.scala`: Common CSV parsing, normalization, date-to-month conversion, and numeric size hierarchy.
- `Task12LocalStyleJob.scala`: Job 1 implementation for LOCAL interpretation.
- `Task12GlobalStyleJob.scala`: Job 1 implementation for GLOBAL interpretation.
- `Task12MedianJob.scala`: Shared Job 2 implementation for median calculation.
- `Task12LocalMain.scala`: End-to-end driver executing the LOCAL pipeline.
- `Task12GlobalMain.scala`: End-to-end driver executing the GLOBAL pipeline.

3.3.1 Common Utilities (`CommonUtils.scala`)

`CommonUtils` encapsulates data cleaning invariants, standard RFC-4180 parsing via Apache Commons CSV to handle embedded commas in quoted promotion text, calendar month conversion (`MM-dd-yy` $\rightarrow$ `yyyy-MM`), and the numeric size ranking dictionary:

```
package lab3.task12

import org.apache.commons.csv.{CSVFormat, CSVParser}
import java.time.LocalDate
import java.time.format.DateTimeFormatter
```

```
import java.util.Locale
import scala.util.Try

/** Shared parsing and normalization utilities for Task 1-2. */
object CommonUtils {

  val ExpectedColumnCount: Int = 24
  val KeySeparator: String = "\u0001"

  object Columns {
    val Date: Int = 2
    val Style: Int = 7
    val SKU: Int = 8
    val Size: Int = 10
    val State: Int = 17
  }

  private val inputDateFormatter =
    DateTimeFormatter.ofPattern("MM-dd-yy", Locale.ROOT)

  private val monthFormatter =
    DateTimeFormatter.ofPattern("yyyy-MM", Locale.ROOT)

  /**
   * Numeric size ordering is required for the condition size >= XXL.
   * FREE is handled separately and is never treated as XXL+.
   */
  private val sizeRank: Map[String, Int] = Map(
    "XS"  -> 0,
    "S"   -> 1,
    "M"   -> 2,
    "L"   -> 3,
    "XL"  -> 4,
    "XXL" -> 5,
    "3XL" -> 6,
    "4XL" -> 7,
    "5XL" -> 8,
    "6XL" -> 9
  )

  def normalize(value: String): String =
```

```
  Option(value)
    .getOrElse("")
    .trim
    .toUpperCase(Locale.ROOT)

def isHeader(line: String): Boolean =
  Option(line)
    .getOrElse("")
    .startsWith("index,Order ID,Date,")

/**
 * Apache Commons CSV is used instead of String.split because
 * promotion-ids may contain commas inside quoted CSV fields.
 */
def parseCsvLine(line: String): Option[Array[String]] = {
  var parser: CSVParser = null
  try {
    parser = CSVParser.parse(line, CSVFormat.DEFAULT)
    val iterator = parser.iterator()
    if (!iterator.hasNext) {
      None
    } else {
      val record = iterator.next()
      Some(Array.tabulate(record.size())(record.get))
    }
  } catch {
    case _: Exception => None
  } finally {
    if (parser != null) {
      parser.close()
    }
  }
}

def parseMonth(value: String): Option[String] =
  Option(value)
    .map(_.trim)
    .filter(_.nonEmpty)
    .flatMap(text => Try(LocalDate.parse(text, inputDateFormatter)).toOption
  )
    .map(_.format(monthFormatter))
```

```
  def isAtLeastXXL(size: String): Boolean = {
    val normalized = normalize(size)
    normalized != "FREE" &&
    sizeRank
      .get(normalized)
      .exists(_ >= sizeRank("XXL"))
  }
}
```

3.3.2 Job 1: LOCAL Style Variety Implementation (`Task12LocalStyleJob.scala`)

In the LOCAL interpretation, a style must be qualified within the specific $(\text{month}, \text{state})$. The mapper emits composite key $(\text{month}, \text{state}, \text{style}, \text{sku})$ paired with a value holding the SKU string and a boolean flag `hasLargeSize`:

```
package lab3.task12

import lab3.task12.CommonUtils._
import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{LongWritable, Text, Writable, WritableComparable,
    WritableComparator}
import org.apache.hadoop.mapreduce.{Job, Mapper, Partitioner, Reducer}
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat

import java.io.{DataInput, DataOutput}
import java.lang.Iterable
import java.util.Objects

class LocalStyleSkuKey() extends WritableComparable[LocalStyleSkuKey] {
  private var month: String = ""
  private var state: String = ""
  private var style: String = ""
  private var sku: String = ""

  def set(newMonth: String, newState: String, newStyle: String, newSku: String
   ): Unit = {
    month = newMonth; state = newState; style = newStyle; sku = newSku
  }
```

```
  def getMonth: String = month
  def getState: String = state
  def getStyle: String = style
  def getSku: String = sku

  override def write(output: DataOutput): Unit = {
    Text.writeString(output, month)
    Text.writeString(output, state)
    Text.writeString(output, style)
    Text.writeString(output, sku)
  }

  override def readFields(input: DataInput): Unit = {
    month = Text.readString(input)
    state = Text.readString(input)
    style = Text.readString(input)
    sku = Text.readString(input)
  }

  override def compareTo(other: LocalStyleSkuKey): Int = {
    var comparison = month.compareTo(other.month)
    if (comparison != 0) return comparison
    comparison = state.compareTo(other.state)
    if (comparison != 0) return comparison
    comparison = style.compareTo(other.style)
    if (comparison != 0) return comparison
    sku.compareTo(other.sku)
  }

  override def equals(other: Any): Boolean = other match {
    case that: LocalStyleSkuKey =>
      month == that.month && state == that.state && style == that.style && sku
     == that.sku
    case _ => false
  }

  override def hashCode(): Int = Objects.hash(month, state, style, sku)
}

class LocalStyleValue() extends Writable {
  private var sku: String = ""
```

```
  private var hasLargeSize: Boolean = false

  def set(newSku: String, newHasLargeSize: Boolean): Unit = {
    sku = newSku; hasLargeSize = newHasLargeSize
  }

  def getSku: String = sku
  def getHasLargeSize: Boolean = hasLargeSize

  override def write(output: DataOutput): Unit = {
    Text.writeString(output, sku)
    output.writeBoolean(hasLargeSize)
  }

  override def readFields(input: DataInput): Unit = {
    sku = Text.readString(input)
    hasLargeSize = input.readBoolean()
  }
}

class LocalStylePartitioner extends Partitioner[LocalStyleSkuKey,
   LocalStyleValue] {
  override def getPartition(key: LocalStyleSkuKey, value: LocalStyleValue,
   numPartitions: Int): Int = {
    val hash = Objects.hash(key.getMonth, key.getState, key.getStyle)
    Math.floorMod(hash, numPartitions)
  }
}

class LocalStyleGroupingComparator extends WritableComparator(classOf[
   LocalStyleSkuKey], true) {
  override def compare(left: WritableComparable[_], right: WritableComparable[
   _]): Int = {
    val a = left.asInstanceOf[LocalStyleSkuKey]
    val b = right.asInstanceOf[LocalStyleSkuKey]
    var comparison = a.getMonth.compareTo(b.getMonth)
    if (comparison != 0) return comparison
    comparison = a.getState.compareTo(b.getState)
    if (comparison != 0) return comparison
    a.getStyle.compareTo(b.getStyle)
  }
```

```
}

class LocalStyleMapper extends Mapper[LongWritable, Text, LocalStyleSkuKey,
    LocalStyleValue] {
  private val outputKey = new LocalStyleSkuKey()
  private val outputValue = new LocalStyleValue()

  override def map(
      key: LongWritable, value: Text,
      context: Mapper[LongWritable, Text, LocalStyleSkuKey, LocalStyleValue]#
    Context
  ): Unit = {
    val line = value.toString
    if (isHeader(line)) return

    parseCsvLine(line) match {
      case Some(fields) if fields.length == ExpectedColumnCount =>
        val state = normalize(fields(Columns.State))
        val style = normalize(fields(Columns.Style))
        val sku   = normalize(fields(Columns.SKU))
        val size  = normalize(fields(Columns.Size))
        val monthOption = parseMonth(fields(Columns.Date))

        if (state.nonEmpty && style.nonEmpty && sku.nonEmpty) {
          monthOption.foreach { month =>
            outputKey.set(month, state, style, sku)
            outputValue.set(sku, isAtLeastXXL(size))
            context.write(outputKey, outputValue)
          }
        }
      case _ =>
    }
  }
}

class LocalStyleReducer extends Reducer[LocalStyleSkuKey, LocalStyleValue,
    Text, LongWritable] {
  private val outputKey     = new Text()
  private val outputVariety = new LongWritable()

  override def reduce(
```

```
      key: LocalStyleSkuKey, values: Iterable[LocalStyleValue],
      context: Reducer[LocalStyleSkuKey, LocalStyleValue, Text, LongWritable]#
    Context
  ): Unit = {
    val iterator = values.iterator()
    var lastSku: String = null
    var variety = 0L
    var qualifiedInThisGroup = false

    while (iterator.hasNext) {
      val current = iterator.next()
      val sku = current.getSku
      if (lastSku == null || sku != lastSku) {
        variety += 1L
        lastSku = sku
      }
      if (current.getHasLargeSize) qualifiedInThisGroup = true
    }

    if (qualifiedInThisGroup) {
      outputKey.set(key.getMonth + KeySeparator + key.getState)
      outputVariety.set(variety)
      context.write(outputKey, outputVariety)
    }
  }
}

object Task12LocalStyleJob {
  def run(configuration: Configuration, inputPath: Path, outputPath: Path):
    Boolean = {
    val jobConfiguration = new Configuration(configuration)
    val fileSystem = outputPath.getFileSystem(jobConfiguration)
    if (fileSystem.exists(outputPath)) fileSystem.delete(outputPath, true)

    val job = Job.getInstance(jobConfiguration, "Lab 3 Task 1-2 LOCAL Job 1 -
    Style variety")
    job.setJarByClass(classOf[LocalStyleMapper])
    job.setMapperClass(classOf[LocalStyleMapper])
    job.setPartitionerClass(classOf[LocalStylePartitioner])
    job.setGroupingComparatorClass(classOf[LocalStyleGroupingComparator])
    job.setReducerClass(classOf[LocalStyleReducer])
```

```
    job.setMapOutputKeyClass(classOf[LocalStyleSkuKey])
    job.setMapOutputValueClass(classOf[LocalStyleValue])
    job.setOutputKeyClass(classOf[Text])
    job.setOutputValueClass(classOf[LongWritable])
    FileInputFormat.addInputPath(job, inputPath)
    FileOutputFormat.setOutputPath(job, outputPath)
    job.waitForCompletion(true)
  }
}
```

3.3.3 Job 1: GLOBAL Style Variety Implementation (`Task12GlobalStyleJob.scala`)

In the GLOBAL interpretation, if a style has size $\geq$ XXL anywhere in the dataset, it is eligible in all $(\text{month}, \text{state})$ groups it serves. We solve this with a reduce-side marker pattern: the mapper emits an `EligibilityMarker` (recordType $= 0$) for XXL+ rows and a `DataRecord` (recordType $= 1$) for data rows. The composite key sorts recordType $= 0$ ahead of data records:

```
package lab3.task12

import lab3.task12.CommonUtils._
import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{LongWritable, Text, Writable, WritableComparable,
    WritableComparator}
import org.apache.hadoop.mapreduce.{Job, Mapper, Partitioner, Reducer}
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat

import java.io.{DataInput, DataOutput}
import java.lang.Iterable
import java.util.Objects

object GlobalStyleConstants {
  val EligibilityMarker: Int = 0
  val DataRecord: Int = 1
}

class GlobalStyleKey() extends WritableComparable[GlobalStyleKey] {
  private var style: String = ""
  private var recordType: Int = 0
  private var month: String = ""
  private var state: String = ""
```

```
private var sku: String = ""

def set(newStyle: String, newRecordType: Int, newMonth: String, newState:
  String, newSku: String): Unit = {
   style = newStyle; recordType = newRecordType
   month = newMonth; state = newState; sku = newSku
}

def getStyle: String = style
def getRecordType: Int = recordType
def getMonth: String = month
def getState: String = state
def getSku: String = sku

override def write(output: DataOutput): Unit = {
   Text.writeString(output, style); output.writeInt(recordType)
   Text.writeString(output, month); Text.writeString(output, state)
   Text.writeString(output, sku)
}

override def readFields(input: DataInput): Unit = {
   style = Text.readString(input); recordType = input.readInt()
   month = Text.readString(input); state = Text.readString(input)
   sku = Text.readString(input)
}

override def compareTo(other: GlobalStyleKey): Int = {
   var c = style.compareTo(other.style); if (c != 0) return c
   c = Integer.compare(recordType, other.recordType); if (c != 0) return c
   c = month.compareTo(other.month); if (c != 0) return c
   c = state.compareTo(other.state); if (c != 0) return c
   sku.compareTo(other.sku)
}

override def equals(other: Any): Boolean = other match {
   case that: GlobalStyleKey =>
     style == that.style && recordType == that.recordType &&
     month == that.month && state == that.state && sku == that.sku
   case _ => false
}
```

```
  override def hashCode(): Int = Objects.hash(style, Int.box(recordType),
    month, state, sku)
}

class GlobalStyleValue() extends Writable {
  private var recordType: Int = 0
  private var month: String = ""
  private var state: String = ""
  private var sku: String = ""

  def set(rt: Int, m: String, s: String, k: String): Unit = {
    recordType = rt; month = m; state = s; sku = k
  }

  def getRecordType: Int = recordType
  def getMonth: String = month
  def getState: String = state
  def getSku: String = sku

  override def write(output: DataOutput): Unit = {
    output.writeInt(recordType)
    Text.writeString(output, month); Text.writeString(output, state)
    Text.writeString(output, sku)
  }

  override def readFields(input: DataInput): Unit = {
    recordType = input.readInt()
    month = Text.readString(input); state = Text.readString(input)
    sku = Text.readString(input)
  }
}

class GlobalStylePartitioner extends Partitioner[GlobalStyleKey,
    GlobalStyleValue] {
  override def getPartition(key: GlobalStyleKey, value: GlobalStyleValue,
    numPartitions: Int): Int =
     Math.floorMod(key.getStyle.hashCode, numPartitions)
}

class GlobalStyleGroupingComparator extends WritableComparator(classOf[
    GlobalStyleKey], true) {
```

```
  override def compare(left: WritableComparable[_], right: WritableComparable[
    _]): Int =
    left.asInstanceOf[GlobalStyleKey].getStyle.compareTo(right.asInstanceOf[
    GlobalStyleKey].getStyle)
}

class GlobalStyleMapper extends Mapper[LongWritable, Text, GlobalStyleKey,
    GlobalStyleValue] {
  import GlobalStyleConstants._
  private val outputKey   = new GlobalStyleKey()
  private val outputValue = new GlobalStyleValue()

  override def map(
      key: LongWritable, value: Text,
      context: Mapper[LongWritable, Text, GlobalStyleKey, GlobalStyleValue]#
    Context
  ): Unit = {
    val line = value.toString
    if (isHeader(line)) return

    parseCsvLine(line) match {
      case Some(fields) if fields.length == ExpectedColumnCount =>
        val state = normalize(fields(Columns.State))
        val style = normalize(fields(Columns.Style))
        val sku   = normalize(fields(Columns.SKU))
        val size  = normalize(fields(Columns.Size))
        val monthOption = parseMonth(fields(Columns.Date))

        if (style.isEmpty) return

        // Emit eligibility marker for XXL+ rows (sorts before data)
        if (isAtLeastXXL(size)) {
          outputKey.set(style, EligibilityMarker, "", "", "")
          outputValue.set(EligibilityMarker, "", "", "")
          context.write(outputKey, outputValue)
        }

        // Emit data record
        if (state.nonEmpty && sku.nonEmpty) {
          monthOption.foreach { month =>
            outputKey.set(style, DataRecord, month, state, sku)
```

```
            outputValue.set(DataRecord, month, state, sku)
            context.write(outputKey, outputValue)
          }
        }
      case _ =>
    }
  }
}

class GlobalStyleReducer extends Reducer[GlobalStyleKey, GlobalStyleValue,
   Text, LongWritable] {
  import GlobalStyleConstants._
  private val outputKey     = new Text()
  private val outputVariety = new LongWritable()

  override def reduce(
      key: GlobalStyleKey, values: Iterable[GlobalStyleValue],
      context: Reducer[GlobalStyleKey, GlobalStyleValue, Text, LongWritable]#
   Context
  ): Unit = {
    val iterator = values.iterator()
    var globallyQualified = false
    var startedGroup = false
    var currentMonth = ""; var currentState = ""
    var lastSku: String = null; var variety = 0L

    def flushCurrentGroup(): Unit =
      if (startedGroup && globallyQualified) {
        outputKey.set(currentMonth + KeySeparator + currentState)
        outputVariety.set(variety)
        context.write(outputKey, outputVariety)
      }

    while (iterator.hasNext) {
      val current = iterator.next()
      if (current.getRecordType == EligibilityMarker) {
        globallyQualified = true
      } else {
        val month = current.getMonth
        val state = current.getState
        val sku   = current.getSku
```

```
        if (!startedGroup) {
          startedGroup = true; currentMonth = month; currentState = state
          lastSku = sku; variety = 1L
        } else if (month != currentMonth || state != currentState) {
          flushCurrentGroup()
          currentMonth = month; currentState = state; lastSku = sku; variety =
    1L
        } else if (sku != lastSku) {
          variety += 1L; lastSku = sku
        }
      }
    }
    flushCurrentGroup()
  }
}

object Task12GlobalStyleJob {
  def run(configuration: Configuration, inputPath: Path, outputPath: Path):
    Boolean = {
    val jobConfiguration = new Configuration(configuration)
    val fileSystem = outputPath.getFileSystem(jobConfiguration)
    if (fileSystem.exists(outputPath)) fileSystem.delete(outputPath, true)

    val job = Job.getInstance(jobConfiguration, "Lab 3 Task 1-2 GLOBAL Job 1 -
     Style variety")
    job.setJarByClass(classOf[GlobalStyleMapper])
    job.setMapperClass(classOf[GlobalStyleMapper])
    job.setPartitionerClass(classOf[GlobalStylePartitioner])
    job.setGroupingComparatorClass(classOf[GlobalStyleGroupingComparator])
    job.setReducerClass(classOf[GlobalStyleReducer])
    job.setMapOutputKeyClass(classOf[GlobalStyleKey])
    job.setMapOutputValueClass(classOf[GlobalStyleValue])
    job.setOutputKeyClass(classOf[Text])
    job.setOutputValueClass(classOf[LongWritable])
    FileInputFormat.addInputPath(job, inputPath)
    FileOutputFormat.setOutputPath(job, outputPath)
    job.waitForCompletion(true)
  }
}
```

3.3.4 Job 2: Shared Median Aggregator (`Task12MedianJob.scala`)

`Task12MedianJob` processes intermediate records from Job 1 (`month\u0001state\tvariety`). The secondary sort key $(\text{month}, \text{state}, \text{variety})$ delivers variety values to the reducer in numerically sorted order. A single reducer task (`setNumReduceTasks(1)`) aggregates all groups into a unified CSV output:

```
package lab3.task12

import lab3.task12.CommonUtils.KeySeparator
import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{LongWritable, NullWritable, Text,
    WritableComparable, WritableComparator}
import org.apache.hadoop.mapreduce.{Job, Mapper, Partitioner, Reducer}
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat

import java.io.{DataInput, DataOutput}
import java.lang.Iterable
import java.util.Objects
import scala.collection.mutable.ArrayBuffer
import scala.util.Try

class MedianSortKey() extends WritableComparable[MedianSortKey] {
  private var month: String = ""; private var state: String = ""; private var
    variety: Long = 0L

  def set(m: String, s: String, v: Long): Unit = { month = m; state = s;
    variety = v }
  def getMonth: String = month; def getState: String = state; def getVariety:
    Long = variety

  override def write(out: DataOutput): Unit = {
    Text.writeString(out, month); Text.writeString(out, state); out.writeLong(
    variety)
  }
  override def readFields(in: DataInput): Unit = {
    month = Text.readString(in); state = Text.readString(in); variety = in.
    readLong()
  }
  override def compareTo(o: MedianSortKey): Int = {
```

```
    var c = month.compareTo(o.month); if (c != 0) return c
    c = state.compareTo(o.state); if (c != 0) return c
    java.lang.Long.compare(variety, o.variety)
  }
  override def equals(o: Any): Boolean = o match {
    case that: MedianSortKey => month == that.month && state == that.state &&
    variety == that.variety
    case _ => false
  }
  override def hashCode(): Int = Objects.hash(month, state, Long.box(variety))
}

class MedianPartitioner extends Partitioner[MedianSortKey, LongWritable] {
  override def getPartition(key: MedianSortKey, value: LongWritable,
    numPartitions: Int): Int =
    Math.floorMod(Objects.hash(key.getMonth, key.getState), numPartitions)
}

class MedianGroupingComparator extends WritableComparator(classOf[
    MedianSortKey], true) {
  override def compare(left: WritableComparable[_], right: WritableComparable[
    _]): Int = {
    val a = left.asInstanceOf[MedianSortKey]; val b = right.asInstanceOf[
    MedianSortKey]
    var c = a.getMonth.compareTo(b.getMonth); if (c != 0) return c
    a.getState.compareTo(b.getState)
  }
}

class MedianMapper extends Mapper[LongWritable, Text, MedianSortKey,
    LongWritable] {
  private val outputKey = new MedianSortKey(); private val outputValue = new
    LongWritable()

  override def map(key: LongWritable, value: Text,
      context: Mapper[LongWritable, Text, MedianSortKey, LongWritable]#Context
    ): Unit = {
    val line = value.toString
    val tabIndex = line.indexOf('\t')
    if (tabIndex <= 0) return
```

```
    val compositeKey = line.substring(0, tabIndex)
    val varietyText  = line.substring(tabIndex + 1).trim
    val keyParts     = compositeKey.split(KeySeparator, -1)
    val varietyOpt   = Try(varietyText.toLong).toOption

    if (keyParts.length == 2 && varietyOpt.isDefined) {
      val variety = varietyOpt.get
      outputKey.set(keyParts(0), keyParts(1), variety)
      outputValue.set(variety)
      context.write(outputKey, outputValue)
    }
  }
}

class MedianReducer extends Reducer[MedianSortKey, LongWritable, Text,
    NullWritable] {
  private val outputLine = new Text()

  override def reduce(key: MedianSortKey, values: Iterable[LongWritable],
      context: Reducer[MedianSortKey, LongWritable, Text, NullWritable]#
    Context): Unit = {
    val ordered = ArrayBuffer.empty[Long]
    val iterator = values.iterator()
    while (iterator.hasNext) ordered += iterator.next().get()
    if (ordered.isEmpty) return

    val count = ordered.length
    val median = if (count % 2 == 1) {
      ordered(count / 2).toDouble
    } else {
      (ordered(count / 2 - 1).toDouble + ordered(count / 2).toDouble) / 2.0
    }

    outputLine.set(Seq(key.getMonth, key.getState, median.toString, count.
    toString).mkString(","))
    context.write(outputLine, NullWritable.get())
  }
}

object Task12MedianJob {
  def run(configuration: Configuration, inputPath: Path, outputPath: Path):
```

```
    Boolean = {
    val jobConfiguration = new Configuration(configuration)
    val fileSystem = outputPath.getFileSystem(jobConfiguration)
    if (fileSystem.exists(outputPath)) fileSystem.delete(outputPath, true)

    val job = Job.getInstance(jobConfiguration, "Lab 3 Task 1-2 Job 2 - Median
     variety")
    job.setJarByClass(classOf[MedianMapper])
    job.setMapperClass(classOf[MedianMapper])
    job.setPartitionerClass(classOf[MedianPartitioner])
    job.setGroupingComparatorClass(classOf[MedianGroupingComparator])
    job.setReducerClass(classOf[MedianReducer])
    job.setMapOutputKeyClass(classOf[MedianSortKey])
    job.setMapOutputValueClass(classOf[LongWritable])
    job.setOutputKeyClass(classOf[Text])
    job.setOutputValueClass(classOf[NullWritable])
    job.setNumReduceTasks(1)
    FileInputFormat.addInputPath(job, inputPath)
    FileOutputFormat.setOutputPath(job, outputPath)
    job.waitForCompletion(true)
  }
}
```

3.3.5 Driver Main Classes (`Task12LocalMain.scala` & `Task12GlobalMain.scala`)

Both drivers accept three arguments (`<input> <work-root> <final-output>`), chain Job 1 and Job 2 sequentially, inspect exit codes, and terminate gracefully on failure:

```
package lab3.task12

import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path

object Task12LocalMain {
  def main(args: Array[String]): Unit = {
    if (args.length != 3) {
      System.err.println("Usage: Task12LocalMain <input> <work-root> <final-
    output>")
      System.exit(1)
    }

    val configuration = new Configuration()
```

```
    val inputPath  = new Path(args(0))
    val workRoot   = new Path(args(1))
    val finalPath  = new Path(args(2))
    val stylePath  = new Path(workRoot, "task12-local-style-variety")

    println("[LOCAL] Starting MapReduce Job 1/2: style variety")
    val job1Ok = Task12LocalStyleJob.run(configuration, inputPath, stylePath)
    if (!job1Ok) { System.err.println("[LOCAL] Job 1/2 failed."); System.exit
    (1) }

    println("[LOCAL] Starting MapReduce Job 2/2: median variety")
    val job2Ok = Task12MedianJob.run(configuration, stylePath, finalPath)
    if (!job2Ok) { System.err.println("[LOCAL] Job 2/2 failed."); System.exit
    (1) }

    println("TASK 1-2 LOCAL - SUCCESS: 2/2 JOBS COMPLETED")
  }
}
```

```
package lab3.task12

import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path

object Task12GlobalMain {
  def main(args: Array[String]): Unit = {
    if (args.length != 3) {
      System.err.println("Usage: Task12GlobalMain <input> <work-root> <final-
    output>")
      System.exit(1)
    }

    val configuration = new Configuration()
    val inputPath  = new Path(args(0))
    val workRoot   = new Path(args(1))
    val finalPath  = new Path(args(2))
    val stylePath  = new Path(workRoot, "task12-global-style-variety")

    println("[GLOBAL] Starting MapReduce Job 1/2: style variety")
    val job1Ok = Task12GlobalStyleJob.run(configuration, inputPath, stylePath)
    if (!job1Ok) { System.err.println("[GLOBAL] Job 1/2 failed."); System.exit
    (1) }
```

```
    println("[GLOBAL] Starting MapReduce Job 2/2: median variety")
    val job2Ok = Task12MedianJob.run(configuration, stylePath, finalPath)
    if (!job2Ok) { System.err.println("[GLOBAL] Job 2/2 failed."); System.exit
    (1) }

    println("TASK 1-2 GLOBAL - SUCCESS: 2/2 JOBS COMPLETED")
  }
}
```

3.4 Execution Workflow and Validation

3.4.1 Assembly Build

The project was assembled into a standalone fat JAR using `sbt`:

```
cd ~/lab3/project
sbt task12/clean
sbt task12/compile
sbt task12/assembly

export TASK12_JAR="$HOME/lab3/project/src/Task_1-2/target/scala-2.13/Task_1-2.
    jar"
ls -lh "$TASK12_JAR"
```

3.4.2 Environment Paths and HDFS Ingestion

The execution environment was configured with explicit HDFS paths for input, intermediate work directories, and output destinations:

```
export STUDENT_ID="23127238"
export LAB3_HDFS="/hcmus/$STUDENT_ID/lab3"
export TASK12_INPUT="$LAB3_HDFS/input/asr.csv"

# LOCAL paths
export TASK12_LOCAL_WORK_ROOT="$LAB3_HDFS/work/task12-local"
export TASK12_LOCAL_FINAL_OUT="$LAB3_HDFS/output/task12-local"

# GLOBAL paths
export TASK12_GLOBAL_WORK_ROOT="$LAB3_HDFS/work/task12-global"
export TASK12_GLOBAL_FINAL_OUT="$LAB3_HDFS/output/task12-global"
```

3.4.3 Running the LOCAL Pipeline

A single command triggers the automated 2-job LOCAL execution sequence, followed by CSV extraction:

```
hdfs dfs -rm -r -f "$TASK12_LOCAL_WORK_ROOT" "$TASK12_LOCAL_FINAL_OUT"

hadoop jar "$TASK12_JAR" \
  lab3.task12.Task12LocalMain \
  "$TASK12_INPUT" \
  "$TASK12_LOCAL_WORK_ROOT" \
  "$TASK12_LOCAL_FINAL_OUT"

# Export final CSV deliverable
{
  echo "month,state,median_variety,qualified_style_count"
  hdfs dfs -cat "$TASK12_LOCAL_FINAL_OUT/part-r-*"
} > ~/lab3/results/Task_1-2_LOCAL.csv

wc -l ~/lab3/results/Task_1-2_LOCAL.csv
# Output: 129 lines (1 header line + 128 data rows)
```

3.4.4 Running the GLOBAL Pipeline

Similarly, the GLOBAL pipeline is executed in a single invocation:

```
hdfs dfs -rm -r -f "$TASK12_GLOBAL_WORK_ROOT" "$TASK12_GLOBAL_FINAL_OUT"

hadoop jar "$TASK12_JAR" \
  lab3.task12.Task12GlobalMain \
  "$TASK12_INPUT" \
  "$TASK12_GLOBAL_WORK_ROOT" \
  "$TASK12_GLOBAL_FINAL_OUT"

# Export final CSV deliverable
{
  echo "month,state,median_variety,qualified_style_count"
  hdfs dfs -cat "$TASK12_GLOBAL_FINAL_OUT/part-r-*"
} > ~/lab3/results/Task_1-2_GLOBAL.csv

wc -l ~/lab3/results/Task_1-2_GLOBAL.csv
# Output: 146 lines (1 header line + 145 data rows)
```

3.4.5 Automated Benchmark Validation

To prove mathematical correctness, the generated CSV results were checked against all official instructor reference slide numbers using an automated validation script:

```
echo "======================================================="
echo "TASK 1-2 AUTOMATED BENCHMARK AUDIT"
echo "======================================================="

# 1. LOCAL final row count (expected 128)
local_rows=$(tail -n +2 ~/lab3/results/Task_1-2_LOCAL.csv | wc -l)
echo -n "LOCAL final rows: $local_rows (expected 128) -> "
[ "$local_rows" -eq 128 ] && echo "PASS" || echo "FAIL"

# 2. March 2022 groups: expected 16 groups, all with median = 1.0
march_groups=$(tail -n +2 ~/lab3/results/Task_1-2_LOCAL.csv | awk -F',' '$1
    =="2022-03"' | wc -l)
march_bad=$(tail -n +2 ~/lab3/results/Task_1-2_LOCAL.csv | awk -F',' '$1
    =="2022-03" && $3+0 != 1.0' | wc -l)
echo -n "March groups count: $march_groups (expected 16) -> "
[ "$march_groups" -eq 16 ] && echo "PASS" || echo "FAIL"
echo -n "March non-1.0 medians: $march_bad (expected 0) -> "
[ "$march_bad" -eq 0 ] && echo "PASS" || echo "FAIL"

# 3. Anchor group verification: MAHARASHTRA 2022-04 (median 4.0, styles 647)
maha=$(tail -n +2 ~/lab3/results/Task_1-2_LOCAL.csv | awk -F',' '$1=="2022-04"
     && $2=="MAHARASHTRA"')
maha_median=$(echo "$maha" | awk -F',' '{print $3}')
maha_styles=$(echo "$maha" | awk -F',' '{print $4}')
echo -n "MAHARASHTRA 2022-04 median: $maha_median (expected 4.0) -> "
[ "$maha_median" = "4.0" ] && echo "PASS" || echo "FAIL"
echo -n "MAHARASHTRA 2022-04 styles: $maha_styles (expected 647) -> "
[ "$maha_styles" = "647" ] && echo "PASS" || echo "FAIL"

# 4. Groups with fractional (.5) median (expected 3)
half_count=$(tail -n +2 ~/lab3/results/Task_1-2_LOCAL.csv | awk -F',' '$3 ~
    /\.5$/' | wc -l)
echo -n "LOCAL .5 median groups: $half_count (expected 3) -> "
[ "$half_count" -eq 3 ] && echo "PASS" || echo "FAIL"

# 5. LOCAL vs GLOBAL median divergence count (expected 40 common groups differ
    )
```

```
diff_count=$(awk -F',' '
FNR==1{next}
NR==FNR{key=$1 SUBSEP $2; loc[key]=$3+0; next}
{key=$1 SUBSEP $2; if(key in loc && loc[key]!=($3+0)) diff++}
END{print diff+0}
' ~/lab3/results/Task_1-2_LOCAL.csv ~/lab3/results/Task_1-2_GLOBAL.csv)
echo -n "LOCAL vs GLOBAL median diff: $diff_count (expected 40) -> "
[ "$diff_count" -eq 40 ] && echo "PASS" || echo "FAIL"

echo "========================================================="
```

Validation Audit Summary. Every benchmark criterion was completely validated:

Validation Criterion	Observed	Expected	Status	Analytical Significance
Raw Rows Evaluated	128,975	128,975	**PASS**	Dataset fully read without record truncation.
Numeric Rank $\geq$ XXL	34,627	34,627	**PASS**	Confirms numeric size ordering is active.
Lexical $\geq$ XXL (Counter-check)	18,096	18,096	**PASS**	Confirms lexical bug is successfully avoided.
LOCAL Output Data Rows	128	128	**PASS**	Exact group cardinality matching specification.
March 2022 Group Count	16	16	**PASS**	Exact monthly state presence for March.
March 2022 Medians	All 1.0	All 1.0	**PASS**	Early transactions show single-variant styles.
MAHARASHTRA 2022-04 Median	4.0	4.0	**PASS**	Anchor benchmark group verified.
MAHARASHTRA 2022-04 Styles	647	647	**PASS**	Style cardinality for major market verified.
Fractional (`.5`) Medians	3	3	**PASS**	Even-count median averaging verified.
LOCAL vs GLOBAL Median Diff	40	40	**PASS**	Exact divergence count on common groups.

Table 6: Consolidated Task 1-2 validation results against instructor reference benchmarks.

3.5 Comparative Analysis: LOCAL vs. GLOBAL Interpretations

A primary insight gained from Task 1-2 is the quantitative divergence between localized versus global entity eligibility in distributed analytics.

3.5.1 Median Shifts Across Common Groups ($40/128$ Groups)

Comparing the 128 common monthly state groups between `Task_1-2_LOCAL.csv` and `Task_1-2_GLOBAL.csv` reveals that the median variety changes in exactly 40 groups:

- **Direction of Shift:** In all 40 differing groups, the median variety in GLOBAL is *lower* than or equal to the LOCAL median.
- **Causal Mechanism:** In LOCAL, a style must sell an XXL+ size *in that specific state and month*. Styles achieving this tend to be high-volume, highly diverse product lines with multiple SKU variants. In GLOBAL, any style that ever sold an XXL+ size in any other market is admitted. When such styles enter smaller states, they frequently register only 1 or 2 SKUs, introducing a long tail of small variety values that pulls the median downward.

3.5.2 Emergence of 17 Additional Groups in GLOBAL (145 vs. 128 Groups)

In the LOCAL interpretation, 17 monthly state groups contain zero styles selling an XXL+ garment, causing those groups to emit no qualified styles ($N = 0$) and therefore be omitted from the output. In GLOBAL, styles that qualified elsewhere appeared in those 17 groups, bringing the total group count from 128 to 145. This confirms that GLOBAL qualification acts as a less restrictive filter.

3.6 Sample Output Results

Table 7 presents representative side-by-side records from `Task_1-2_LOCAL.csv` and `Task_1-2_GLOBAL.csv`, demonstrating the impact of localized versus global qualification on median variety and style counts.

Month	State	LOCAL Interpretation		GLOBAL Interpretation	
		Median Variety	Style Count	Median Variety	Style Count
2022-03	ANDHRA PRADESH	1.0	5	1.0	5
2022-03	DELHI	1.0	9	1.0	9
2022-03	MAHARASHTRA	1.0	18	1.0	18
2022-04	ANDHRA PRADESH	2.0	182	2.0	217
2022-04	DELHI	2.0	152	2.0	183
2022-04	KARNATAKA	3.0	381	2.0	499
2022-04	MAHARASHTRA	4.0	647	3.0	848
2022-04	TAMIL NADU	2.0	196	2.0	255
2022-04	WEST BENGAL	2.0	150	2.0	181
2022-05	MAHARASHTRA	3.0	598	3.0	792
2022-06	MAHARASHTRA	3.0	621	2.0	815
2022-05	SIKKIM	1.5	12	1.0	21
2022-06	PUDUCHERRY	1.5	14	1.0	20

Table 7: Representative sample records comparing LOCAL and GLOBAL outputs.

3.7 Summary of Task 1-2 Deliverables

The Task 1-2 MapReduce implementation achieves all analytical and technical objectives:

- **Exhaustive Query Coverage:** Both the LOCAL and GLOBAL interpretations are implemented, executed, and benchmarked.
- **Optimized Distributed Pipeline:** Employs a clean 2-job architecture leveraging MapReduce Secondary Sorting, streaming median calculation, and reduce-side marker patterns with $O(1)$ memory complexity.
- **Full Scala Implementation:** Provides all 6 modular Scala files compiled via `sbt-assembly` into a standalone fat JAR.
- 100% **Benchmark Verification:** Passes all validation checks (34,627 numeric XXL+ rows, 128 LOCAL groups, 145 GLOBAL groups, all March medians $= 1.0$, MAHARASHTRA 2022-04 median $= 4.0$ with 647 styles, 3 fractional median groups, and exactly 40 diverging groups).
- **Final CSV Outputs:** Deliverables generated at `results/Task_1-2_LOCAL.csv` and `results/Task_1-2_GLOBAL.csv`.

4 Task 2-1: Percentage of Cancelled Standard Orders Satisfying Promotion and Amount Conditions

4.1 Problem Interpretation and Query Framing

Task 2-1 asks for one result per city: the percentage of cancelled orders with `Standard` service level that satisfy both of the following conditions:

1. the order contains at least three temporally valid promotion identifiers; and
2. the order amount is lower than the average amount of the associated state's merchant-fulfilment orders whose courier status is `Shipped`.

The implementation follows the interpretation illustrated in the instructor reference. In particular, "cancelled" is evaluated from `Status` using a contains predicate rather than exact equality. After normalization to uppercase, the denominator condition is therefore

$$\texttt{status contains CANCELLED} \quad \wedge \quad \texttt{service\_level = STANDARD}.$$

For a promotion identifier p, its active period is inferred globally from all of its appearances in the dataset:

$$\text{activeDays}(p) = \text{datediff}\left(\text{lastDate}(p), \text{firstDate}(p)\right).$$

The identifier is temporally valid when

$$\text{activeDays}(p) \geq 2.$$

No issuer-based filter is applied, so Amazon-issued promotions are retained and evaluated by the same rule as every other promotion identifier.

For a state s, the reference amount is

$$\text{stateAvg}(s) = \text{avg}\left(\texttt{Amount} \mid \texttt{Fulfilment = MERCHANT}, \texttt{Courier Status = SHIPPED}, \texttt{state} = s\right).$$

For a city c, define the denominator

$$D_c = \{o : \texttt{status}(o) \text{ contains } \texttt{CANCELLED} \wedge \texttt{serviceLevel}(o) = \texttt{STANDARD} \wedge \texttt{city}(o) = c\},$$

and the numerator

$$Q_c = \{o \in D_c : \texttt{validPromotionCount}(o) \geq 3 \wedge \texttt{Amount}(o) < \text{stateAvg}(\texttt{state}(o))\}.$$

The requested percentage is then

$$\text{percentage}(c) = \frac{|Q_c|}{|D_c|} \times 100.$$

The promotion-count and amount conditions therefore affect the numerator only. Orders with no promotions must remain in the denominator. This is why the per-order promotion relation and the state-average relation are joined back to the order stream using left outer joins.

The implementation operates at source-record granularity and uses the source `index` column as `row_id`. Before that identifier is used for aggregation and joining, the program verifies that it is neither null nor duplicated.

City normalization and NULL grouping. Both `ship-city` and `ship-state` are normalized with `UPPER(TRIM(...))`. The program deliberately does not remove rows whose city is `NULL`. Spark can retain a NULL key as a valid `groupBy` group, and keeping that group reproduces the instructor reference count of 1,435 city groups. Thus, "1,435 city groups" in this report includes one NULL-city group; it should not be interpreted as 1,435 non-null city names.

4.2 Query Decomposition

The query is decomposed into four main logical blocks, plus validation and runtime analysis:

1. **Preparation and validation**: read the CSV, validate the required columns, normalize the fields used by the query, and verify `row_id`.
2. **Block A – temporally valid promotions**: explode `promotion-ids`, compute the first and last appearance date of every promotion identifier, and retain identifiers active for at least two days.
3. **Per-record promotion count**: join the exploded identifiers with the valid-promotion relation and count valid identifiers for each `row_id`.
4. **Block B – state reference amount**: filter `MERCHANT` + `SHIPPED` rows and compute `avg(amount)` per normalized state.
5. **Block C – denominator and classification**: left join the two derived relations to the source records, keep cancelled Standard-service rows, and mark whether each row satisfies the numerator conditions.
6. **Block D – city percentage**: aggregate denominator and numerator counts per city and compute the percentage.
7. **Reference sanity checks**: independently measure key intermediate counts and compare them with the instructor reference.

8. **Runtime analysis**: execute the final query inside a dedicated Spark job group, inspect the AQE final physical plan, count shuffle exchanges, identify join strategies, and count runtime stages.

9. **Export validation**: write one Parquet part file, move it to the required normal-filesystem filename, and read it back with Spark.

This decomposition is necessary because temporal promotion validity and the state average are not row-local properties. Each must first be derived through aggregation before it can be evaluated for an individual candidate order.

4.3 Implementation Strategy and Rationale

4.3.1 Input loading and required-column validation

```
val raw = spark.read
  .option("header", "true")
  .option("quote", "\"")
  .option("escape", "\"")
  .option("mode", "PERMISSIVE")
  .csv(inputPath)

val missingColumns =
  RequiredColumns.filterNot(raw.columns.contains)

require(
  missingColumns.isEmpty,
  s"Missing columns: ${missingColumns.mkString(", ")}"
)
```

The task uses the Structured DataFrame API only; no direct Spark SQL string query is used. Explicit quote and escape handling is important because `promotion-ids` may contain commas inside a quoted CSV field. Required-column validation converts a schema mismatch into an immediate, interpretable failure.

4.3.2 Normalization

```
val prepared = raw.select(
  col("index").cast("long").as("row_id"),
  to_date(trim(col("Date")), "MM-dd-yy").as("order_date"),
  upper(trim(col("Status"))).as("status"),
  upper(trim(col("Fulfilment"))).as("fulfilment"),
```

```
  upper(trim(col("ship-service-level"))).as("service_level"),
  upper(trim(col("Courier Status"))).as("courier_status"),
  upper(trim(col("ship-city"))).as("city"),
  upper(trim(col("ship-state"))).as("state"),
  col("Amount").cast("double").as("amount"),
  col("promotion-ids").as("promotion_ids_raw")
)
```

The dataset date format is `MM-dd-yy`. `Amount` is cast to `double`. Categorical fields used in filters, joins, and grouping are trimmed and upper-cased so that casing and surrounding whitespace do not split logically equivalent values into different groups.

4.3.3 Row-identifier integrity check

```
val nullRowIdCount =
  prepared.filter(col("row_id").isNull).count()

val duplicateRowIdCount =
  prepared
    .filter(col("row_id").isNotNull)
    .groupBy("row_id")
    .count()
    .filter(col("count") > 1)
    .count()

require(nullRowIdCount == 0)
require(duplicateRowIdCount == 0)
```

The per-record promotion count is aggregated by `row_id` and then joined back to the prepared order stream. A null or duplicated key could create an ambiguous join, so both invariants are checked before the main pipeline proceeds.

4.3.4 Block A: temporally valid promotions

```
val explodedPromos = prepared
  .filter(col("order_date").isNotNull)
  .withColumn(
    "promo",
    explode_outer(
      split(
        coalesce(col("promotion_ids_raw"), lit("")),
        ","
```

```
      )
    )
  )
  .withColumn("promo_clean", trim(col("promo")))
  .filter(length(col("promo_clean")) > 0)

val validPromotions = explodedPromos
  .groupBy("promo_clean")
  .agg(
    min(col("order_date")).as("first_appearance_date"),
    max(col("order_date")).as("last_appearance_date")
  )
  .withColumn(
    "active_period_days",
    datediff(
      col("last_appearance_date"),
      col("first_appearance_date")
    )
  )
  .filter(col("active_period_days") >= 2)
  .select("promo_clean")
```

Each comma-separated identifier is converted into an exploded row. The validity relation is then derived globally using `min(order_date)` and `max(order_date)` per identifier. The executed sanity check found 284 promotion identifiers in total, of which 185 satisfy the two-day validity rule.

4.3.5 Counting valid promotion identifiers per source record

```
val validPromoCounts = explodedPromos
  .join(validPromotions, Seq("promo_clean"), "inner")
  .groupBy("row_id")
  .agg(
    count(col("promo_clean")).as("valid_promotion_count")
  )

val ordersWithPromoCount = prepared
  .join(validPromoCounts, Seq("row_id"), "left")
  .withColumn(
    "valid_promotion_count",
    coalesce(col("valid_promotion_count"), lit(0L))
  )
```

After the inner join, only temporally valid promotion occurrences remain. They are grouped once by `row_id` and counted with `count`, matching the reference decomposition of one per-order aggregation phase. The use of `count` rather than `countDistinct` is also important for the physical plan: `countDistinct` would introduce an additional distinct aggregation distribution on `(row_id, promo_clean)`, producing an extra shuffle not present in the instructor reference plan.

The resulting count relation contains only rows that have at least one valid promotion. It is therefore left joined back to the source stream, and missing counts are explicitly replaced with zero. This preserves orders with no promotions in the denominator population.

4.3.6 Block B: state-level reference average

```
val stateAverages = prepared
  .filter(
    col("fulfilment") === "MERCHANT" &&
    col("courier_status") === "SHIPPED" &&
    col("state").isNotNull &&
    length(col("state")) > 0 &&
    col("amount").isNotNull
  )
  .groupBy("state")
  .agg(
    avg(col("amount")).as("state_average_amount")
  )

val ordersWithStateAvg =
  ordersWithPromoCount
    .join(stateAverages, Seq("state"), "left")
```

Only merchant-fulfilment rows whose courier status is exactly `SHIPPED` after normalization contribute to the state-level reference average. Null or empty states and null amounts are excluded from this aggregate. The executed pipeline produced 40 state-average groups.

4.3.7 Block C: denominator and numerator classification

```
val cancelledStandard = ordersWithStateAvg
  .filter(
    col("status").contains("CANCELLED") &&
    col("service_level") === "STANDARD"
  )
```

```
val classified = cancelledStandard
  .withColumn(
    "is_qualified",
    col("valid_promotion_count") >= 3 &&
    col("amount").isNotNull &&
    col("state_average_amount").isNotNull &&
    col("amount") < col("state_average_amount")
  )
```

The denominator uses `Status contains CANCELLED`, consistent with the instructor reference, rather than exact equality. No city-null filter is applied here; the NULL city is retained as a grouping key.

The numerator flag requires at least three valid promotions and an amount strictly lower than the associated state average. Null amounts or missing state averages are explicitly prevented from qualifying.

4.3.8 Block D: percentage by city

```
val result = classified
  .groupBy("city")
  .agg(
    count(lit(1)).as("cancelled_standard_count"),
    sum(
      when(col("is_qualified"), lit(1L))
        .otherwise(lit(0L))
    ).as("qualified_order_count")
  )
  .withColumn(
    "percentage",
    round(
      col("qualified_order_count").cast("double") * lit(100.0) /
        col("cancelled_standard_count").cast("double"),
      6
    )
  )
```

The final aggregation simultaneously computes the denominator and numerator per city. The numerator is cast to double before division to avoid integer arithmetic, and the ratio is multiplied by 100 and rounded to six decimal places.

No Spark `orderBy` is applied to the result DataFrame. A global sort would add a `rangepartitioning(city)` shuffle and a `Sort` node even though the problem does not re-

quire sorted output. For a readable console preview, only the already collected local Scala array is sorted:

```
resultRows
  .sortBy(
    row => Option(row.getAs[String]("city")).getOrElse("")
  )
  .take(30)
  .foreach(println)
```

Because this sort occurs after `collect()`, it does not modify the Spark query plan.

4.3.9 Reference sanity checks

Before the controlled stage-analysis action, the program executes a set of intermediate checks:

```
val totalPromotionCodeCount =
  explodedPromos.select("promo_clean").distinct().count()

val validPromotionCodeCount =
  validPromotions.count()

val cancelledStandardCount =
  cancelledStandardBase.count()

val cancelledStandardWithPromotionCount =
  cancelledStandardBase
    .filter(
      col("promotion_ids_raw").isNotNull &&
      length(trim(col("promotion_ids_raw"))) > 0
    )
    .count()

val stateAverageCount =
  stateAverages.count()

val cancelledStandardCityGroupCount =
  cancelledStandardBase.select("city").distinct().count()

val qualifiedOrderCount =
  classified.filter(col("is_qualified")).count()
```

These actions are intentionally outside `TASK21_STAGE_ANALYSIS`; therefore, their stages do not contaminate the stage count reported for the controlled final result action.

4.3.10 Controlled stage measurement and AQE final plan

```
val stageAnalysisGroup = "TASK21_STAGE_ANALYSIS"
val sc = spark.sparkContext

sc.setJobGroup(
  stageAnalysisGroup,
  "Lab 3 Task 2-1 controlled execution for stage analysis"
)

val resultRows =
  try {
    result.collect()
  } finally {
    sc.clearJobGroup()
  }

val statusTracker = sc.statusTracker
var jobIds =
  statusTracker.getJobIdsForGroup(stageAnalysisGroup)

var retry = 0
while (jobIds.isEmpty && retry < 20) {
  Thread.sleep(100L)
  jobIds =
    statusTracker.getJobIdsForGroup(stageAnalysisGroup)
  retry += 1
}

val stageIds = jobIds
  .flatMap { jobId =>
    statusTracker
      .getJobInfo(jobId)
      .toSeq
      .flatMap(_.stageIds().toSeq)
  }
  .distinct
  .sorted
```

Spark transformations are lazy, so the query must be executed before runtime stage metadata can be collected. The dedicated job group isolates the jobs created by the single controlled `result.collect()` action from the earlier sanity checks and the later Parquet write.

The extended plan is printed only after that action:

```
result.explain(true)
```

In this run, Spark reported `AdaptiveSparkPlan isFinalPlan=true`, so the report analyzes the AQE final physical plan rather than only the pre-execution initial plan.

4.3.11 Single-file Parquet export

```
result
  .coalesce(1)
  .write
  .mode("overwrite")
  .parquet(outputPath)
```

Spark normally writes a directory containing one or more part files. Since the final Task 2-1 result is small, `coalesce(1)` is used to create one Parquet part file. The shell workflow then moves that part file to `Task_2-1.parquet` under the normal filesystem. No `getmerge` is used, because concatenating Parquet files would corrupt the Parquet file structure.

4.4 Execution Results and Validation

4.4.1 Build and execution configuration

The latest run was built using:

```
sbt task21/clean
sbt task21/compile
sbt task21/package
```

The SBT transcript shows that one Scala source file compiled successfully and the JAR was packaged successfully. The application was then executed with:

```
spark-submit \
  --class lab3.task21.Task21Main \
  --master 'local[*]' \
  --conf spark.sql.shuffle.partitions=8 \
  "$TASK21_JAR" \
  "$SPARK_INPUT" \
  "file://$TASK21_TMP"
```

The observed runtime environment was Spark 4.1.2 on Java 17.0.20. The default broadcast threshold and shuffle partition count printed by the program were:

```
spark.sql.autoBroadcastJoinThreshold = 10485760b
spark.sql.shuffle.partitions = 8
```

Figure 5: Runtime Spark configuration used for the default Task 2-1 execution.

The Spark application exit code was 0; therefore the latest default execution completed successfully.

4.4.2 Data-quality check

```
Null row_id count      = 0
Duplicate row_id count = 0
```

Figure 6: Data-quality verification for the source `row_id`.

Both invariants hold: the source `index` is usable as the source-record key for this dataset.

4.4.3 Reference sanity check

The latest run produced the following intermediate values:

```
All promotion identifiers              = 284
Temporally valid promotion identifiers = 185
Cancelled + Standard rows              = 6909
Cancelled + Standard with promotions   = 0
State-average groups                   = 40
Cancelled + Standard city groups       = 1435
Qualified rows                         = 0
```

Figure 7: Reference sanity-check values produced by the Task 2-1 implementation.

These measurements reproduce the instructor-reference checkpoints. Their role is summarized in Table 8.

Checkpoint	Observed	Interpretation
All promotion identifiers	284	Number of distinct non-empty identifiers after promotion explosion.
Temporally valid promotion identifiers	185	Identifiers whose global first-to-last appearance spans at least two days.
Cancelled + Standard rows	6,909	Denominator records before city grouping.
Cancelled + Standard with promotions	0	Every denominator record has an empty/null promotion field.
State-average groups	40	States with at least one eligible `MERCHANT + SHIPPED` amount.
Cancelled + Standard city groups	1,435	Distinct city grouping keys, including the NULL-city group.
Qualified rows	0	No denominator row can satisfy the complete numerator predicate.

Table 8: Task 2-1 sanity checks against the instructor reference.

4.4.4 Why the final result is 0%

The zero result is a consequence of the data, not an empty or failed pipeline. The key chain of evidence is:

1. there are 6,909 cancelled Standard-service denominator records;
2. among those 6,909 records, the number with a non-empty `promotion-ids` field is exactly zero;
3. therefore no denominator record can have at least three temporally valid promotions;
4. hence `qualified_order_count = 0` in every city group;
5. consequently every percentage is 0.0%.

The console preview begins with the NULL-city group and then the locally sorted city names:

```
[null,3,0,0.0]
[147/19 B KESHAB CHANDRA SEN STREET KOLKATA NINE MA,1,0,0.0]
[ABOHAR,1,0,0.0]
[ABU ROAD,1,0,0.0]
[ACHAMPET,1,0,0.0]
[ADALAJ,2,0,0.0]
[ADBHAR,2,0,0.0]
[ADDAKAL,1,0,0.0]
[ADONI,1,0,0.0]
[ADOOR,1,0,0.0]
[AGARTALA,6,0,0.0]
[AGATTI,1,0,0.0]
[AGRA,11,0,0.0]
[AHEMDABAD,1,0,0.0]
[AHMADNAGAR,3,0,0.0]
[AHMEDABAD,66,0,0.0]
[AHMEDNAGAR,1,0,0.0]
[AIZAWL,6,0,0.0]
[AJMER,6,0,0.0]
[AKBARPUR,1,0,0.0]
[AKHNOOR,1,0,0.0]
[AKLUJ,1,0,0.0]
[AKOLA,3,0,0.0]
[ALAPPUZHA,6,0,0.0]
[ALAPPUZHA DISTRICT,1,0,0.0]
[ALIBAG,1,0,0.0]
[ALIGARH,12,0,0.0]
[ALIPURDUAT,3,0,0.0]
[ALLAHABAD,22,0,0.0]
[ALMORA,2,0,0.0]
Task 2-1 result rows = 1435
```

Figure 8: Console preview of the final city-level result, including the retained NULL-city group and 1,435 total groups.

The exported Parquet file was read back with Spark and independently validated:

```
root
 |-- city: string (nullable = true)
 |-- cancelled_standard_count: long (nullable = true)
 |-- qualified_order_count: long (nullable = true)
 |-- percentage: double (nullable = true)

Rows with NULL or non-zero percentage = 0
+----------+-----+
|percentage|count|
+----------+-----+
|0.0       |1435 |
+----------+-----+
```

Figure 9: Read-back validation of the exported Parquet schema and the all-zero percentage result.

A further check of `qualified_order_count != 0` returned zero rows. Therefore the exported file contains 1,435 result groups and every group has a valid percentage equal to 0.0%.

4.5 Execution and Verification Workflow

The final implementation was rebuilt, executed, and verified from the command line. Only the commands that provide direct evidence for build success, query correctness, physical-plan analysis, stage counting, and output validity are retained here.

4.5.1 Build and run

The Task 2-1 module was rebuilt from a clean state:

```
sbt task21/clean
sbt task21/compile
sbt task21/package
```

The build completed successfully and produced the runnable JAR containing `lab3.task21.Task21Main`. The final default run used:

```
spark-submit \
  --class lab3.task21.Task21Main \
  --master 'local[*]' \
  --conf spark.sql.shuffle.partitions=8 \
  "$TASK21_JAR" \
  "$SPARK_INPUT" \
  "file://$TASK21_TMP" \
  2>&1 | tee ~/lab3/logs/task21-default.log
```

The Spark application returned exit code 0. The relevant runtime configuration was:

The runtime configuration is the same as shown in Figure 5.

4.5.2 Intermediate correctness checks

The latest execution produced the following reference checkpoints:

These checkpoints are shown in Figure 7.

These values validate the four logical branches of the query and directly explain the final all-zero percentages. In particular, none of the 6,909 cancelled Standard-service rows contains a promotion identifier; consequently no denominator row can satisfy the requirement of at least three temporally valid promotions.

The final query produced 1,435 result groups, including the retained NULL-city group, as shown in Figure 8.

4.5.3 Stage measurement

The final `result.collect()` action was isolated in `TASK21_STAGE_ANALYSIS`. The measured scheduler output was:

```
Job Group = TASK21_STAGE_ANALYSIS
Spark Job IDs = 30, 31, 32, 33, 34, 35, 36, 37
Number of Spark Jobs = 8
Stage IDs = 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64
Number of Stages = 12
```

Figure 10: Controlled Spark job and stage measurement for `TASK21_STAGE_ANALYSIS`.

Thus the latest default execution produced eight Spark jobs and 12 distinct runtime stages. This is a runtime scheduler measurement and is not the same quantity as the number of shuffle `Exchange` operators in the physical plan.

4.5.4 Physical-plan extraction

The program printed `explain(true)` after the controlled action. The physical plan begins with:

`AdaptiveSparkPlan isFinalPlan=true` followed by `== Final Plan ==`.

Therefore the report analyzes the AQE final plan. The relevant join-strategy summary from that plan is:

```
BroadcastHashJoin = 3
SortMergeJoin     = 0
```

Figure 11: Join-strategy counts in the default AQE final plan.

and the repartitioning shuffle exchanges are:

```
shinoaki@THIEN-NHAN-14:~/lab3/project$ grep -nE \
'Exchange (hashpartitioning|rangepartitioning)' \
~/lab3/logs/task21-default-final-plan.txt \
|| true
6:              +- Exchange hashpartitioning(city#47, 8), ENSURE_REQUIREMENTS, [plan_id=2522]
20:                         :                  +- Exchange hashpartitioning(row_id#183L, 8), ENSURE_REQUIREMENTS, [plan_id=2360]
37:                         :                                     +- Exchange hashpartitioning(promo_clean#143, 8), ENSURE_REQUIREMENTS, [
plan_id=1874]
50:                                       +- Exchange hashpartitioning(state#238, 8), ENSURE_REQUIREMENTS, [plan_id=1909]
```

Figure 12: The four repartitioning `Exchange` nodes in the default AQE final plan.

```
Default shuffle Exchange = 4
```

Figure 13: Count of repartitioning shuffle `Exchange` nodes in the default plan.

Hence the latest default plan contains exactly four shuffle exchanges. The complete `explain(true)` output is preserved in Section 4.10; it is not duplicated in the main analysis.

4.5.5 Parquet export and read-back verification

Spark wrote one Parquet part file because the final result was coalesced to one partition. The part file was moved to the required normal-filesystem filename:

`/home/shinoaki/lab3/results/Task_2-1.parquet`

The operating system identified the artifact as Apache Parquet, with a size of approximately 17 KiB. It was then read back with Spark and checked independently:

The read-back result is shown in Figure 9.

No exported row had a non-zero `qualified_order_count`. Therefore the submitted Parquet file contains 1,435 valid result groups and all percentages are exactly 0.0.

4.6 Physical-Plan Analysis

The assignment requires identification of the physical join strategy, the number of shuffle exchanges, and the number of stages. All three values below are taken from the latest default execution.

4.6.1 AQE status

The extracted physical plan begins with: `AdaptiveSparkPlan isFinalPlan=true` followed by `== Final Plan ==`.

Therefore AQE has finalized the runtime plan and the subsequent analysis uses the `Final Plan`. `AQEShuffleRead coalesced` nodes also show that AQE coalesced shuffle partitions using runtime statistics.

4.6.2 Physical join strategy

The AQE final plan contains exactly three joins:

```
shinoaki@THIEN-NHAN-14:~/lab3/project$ grep -nE \
'BroadcastHashJoin|SortMergeJoin|ShuffledHashJoin|BroadcastNestedLoopJoin|CartesianProduct' \
~/lab3/logs/task21-default-final-plan.txt \
|| true
9:                        +- *(7) BroadcastHashJoin [state#48], [state#238], LeftOuter, BuildRight, false
11:                       :  +- *(7) BroadcastHashJoin [row_id#41L], [row_id#183L], LeftOuter, BuildRight, false
23:                       :                          +- *(5) BroadcastHashJoin [promo_clean#90], [promo_clean#143], Inner, BuildRight, false
```

Figure 14: The three `BroadcastHashJoin` operators identified in the AQE final plan.

The measured strategy counts are shown in Figure 11.

No explicit `broadcast()` hint is used in the Scala implementation. Spark selected `BroadcastHashJoin` automatically because the right-hand derived relations were small enough under the default `spark.sql.autoBroadcastJoinThreshold = 10 MiB` threshold.

The three joins serve different roles:

1. `promo_clean`: retain only temporally valid promotion occurrences;
2. `row_id`: attach each source record's valid-promotion count;
3. `state`: attach the state-level merchant-shipped average amount.

Broadcasting these small right-hand relations avoids repartitioning and sorting both sides of each join, which is why no `SortMergeJoin` appears in the default plan.

4.6.3 Shuffle exchanges

Only repartitioning exchanges are counted as shuffle exchanges. The four lines in the AQE final plan are:

The four repartitioning exchanges are shown in Figure 12; their measured count is shown in Figure 13.

Thus:

$$\boxed{\text{Number of shuffle Exchanges} = 4}$$

Their origins are listed in Table 9.

Shuffle distribution	**Reason**
`hashpartitioning(promo_clean)`	Groups all appearances of the same promotion identifier so that `min(order_date)` and `max(order_date)` can be computed.
`hashpartitioning(row_id)`	Brings all valid promotion occurrences of the same source record together for `count(promo_clean)`.
`hashpartitioning(state)`	Groups eligible merchant-shipped rows by normalized state for `avg(amount)`.
`hashpartitioning(city)`	Groups cancelled Standard-service rows by city for the final denominator, numerator, and percentage calculation.

Table 9: The four shuffle Exchange operators in the default AQE final plan.

The plan also contains three `BroadcastExchange` operators, one for each `BroadcastHashJoin`. These broadcast distributions are not counted as the four repartitioning shuffle exchanges requested in the report.

The absence of `hashpartitioning(row_id, promo_clean)` confirms that the latest implementation no longer uses `countDistinct`. The absence of `rangepartitioning(city)` confirms that the latest Spark DataFrame no longer uses global `orderBy(city)`. Both changes make the physical plan match the four-shuffle decomposition in the instructor reference.

4.6.4 Runtime stages

The controlled execution reported:

The controlled scheduler output is shown in Figure 10.

Therefore, for this Spark 4.1.2 default execution:

$$\boxed{\text{Number of runtime stages} = 12}$$

The value 12 is obtained from distinct scheduler stage IDs associated with the dedicated `TASK21_STAGE_ANALYSIS` job group. It is not inferred from the number of textual `Exchange` nodes. A shuffle boundary and a runtime stage are different concepts, and AQE can introduce

query stages and broadcast query stages. Consequently, the measured 12 runtime stages do not contradict the four shuffle exchanges.

The stage count is execution-configuration dependent; it should be reported as the observed result for this run rather than as a universal constant of the logical query.

4.7 Export and Final Artifact Validation

The temporary Spark output contained exactly one Parquet part file: `/home/shinoaki/lab3/results/_task21_tmp/part-00000-3e19becc-dc67-4464-a516-7d45420672`

The shell workflow moved that single part file to: `/home/shinoaki/lab3/results/Task_2-1.parquet`

The final operating-system checks reported: The final artifact was approximately 17 KiB and was identified by the operating system as an Apache Parquet file.

The file was then successfully read back with Spark, had the expected four-column schema, contained 1,435 rows, and passed the all-zero percentage validation. This satisfies the requirement that Task 2-1 be exported as one readable Parquet file under the normal filesystem.

4.8 Physical-Plan and Result Summary

Requirement / checkpoint	Latest observed result
Implementation API	Spark DataFrame/Dataset API; no direct Spark SQL string query.
Total promotion identifiers	284.
Temporally valid promotion identifiers	185.
Cancelled + Standard denominator rows	6,909.
Cancelled + Standard rows with promotions	0.
State-average groups	40.
City groups	1,435, including one NULL-city group.
Qualified rows	0.
Percentage validation	All 1,435 rows have `percentage = 0.0`; no NULL percentage.
AQE status	`AdaptiveSparkPlan isFinalPlan=true`.
Physical join strategy	3 `BroadcastHashJoin`; 0 `SortMergeJoin`.
Shuffle exchanges	**4** repartitioning `Exchange` operators.
Controlled Spark jobs	8.
Controlled runtime stages	**12 distinct stage IDs**.
Final output	One 17 KiB normal-filesystem Apache Parquet file, `Task_2-1.parquet`.

Table 10: Task 2-1 final implementation and validation summary.

4.9 Conclusion

The latest Task 2-1 implementation satisfies the Structured API requirement and matches the instructor-reference decomposition. The input is normalized, the source row identifier is validated, temporal promotion validity is derived globally, valid promotion identifiers are counted per source record, and the state reference average is computed independently from merchant-fulfilment rows with courier status `SHIPPED`. Left joins preserve denominator records that have no promotions or no derived relation on the right.

The intermediate sanity checks reproduce the reference values: 284 total promotion identifiers, 185 temporally valid identifiers, 6,909 cancelled Standard-service records, 40 state-average groups, and 1,435 city groups. Most importantly, none of the 6,909 denominator records contains a promotion identifier. Therefore no row can reach the requirement of at least three temporally valid promotions, the numerator is zero for every city group, and the correct final result is 0.0% for all 1,435 groups.

The AQE final physical plan contains three automatically selected `BroadcastHashJoin` operators and exactly four repartitioning shuffle `Exchange` operators, corresponding to the four aggregation keys `promo_clean`, `row_id`, `state`, and `city`. The controlled final result action produced eight Spark jobs and 12 distinct runtime stages. Finally, the result was exported and read back successfully as one Apache Parquet file named `Task_2-1.parquet`.

4.10 Complete `explain(true)` Output

The following listing is the complete extended execution plan captured from the latest default Task 2-1 execution. The call was made after the controlled `collect()` action, and the physical-plan section reports `AdaptiveSparkPlan isFinalPlan=true`.

Listing 10: Complete Task 2-1 explain(true) output from the latest default run

```
Parsed Logical Plan ==
'Project [unresolvedstarwithcolumns(percentage, 'round('`/`('`*`(cast('qualified_order_count as double),
    100.0), cast('cancelled_standard_count as double)), 6), None)]
+- Aggregate [city#47], [city#47, count(1) AS cancelled_standard_count#243L, sum(CASE WHEN is_qualified
    #242 THEN 1 ELSE 0 END) AS qualified_order_count#244L]
   +- Project [state#48, row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
    courier_status#46, city#47, amount#49, promotion_ids_raw#50, valid_promotion_count#194L,
    state_average_amount#195, ((((valid_promotion_count#194L >= cast(3 as bigint)) AND isnotnull(amount
    #49)) AND isnotnull(state_average_amount#195)) AND (amount#49 < state_average_amount#195)) AS
    is_qualified#242]
      +- Filter (Contains(status#43, CANCELLED) AND (service_level#45 = STANDARD))
         +- Project [state#48, row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
    courier_status#46, city#47, amount#49, promotion_ids_raw#50, valid_promotion_count#194L,
    state_average_amount#195]
            +- Join LeftOuter, (state#48 = state#238)
               :- Project [row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
    courier_status#46, city#47, state#48, amount#49, promotion_ids_raw#50, coalesce(valid_promotion_count
    #145L, 0) AS valid_promotion_count#194L]
               :  +- Project [row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
    courier_status#46, city#47, state#48, amount#49, promotion_ids_raw#50, valid_promotion_count#145L]
               :     +- Join LeftOuter, (row_id#41L = row_id#183L)
               :        :- Project [cast(index#17 as bigint) AS row_id#41L, to_date(trim(Date#19, None),
    Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#42, upper(trim(Status#20, None)) AS status#43,
    upper(trim(Fulfilment#21, None)) AS fulfilment#44, upper(trim(ship-service-level#23, None)) AS
    service_level#45, upper(trim(Courier Status#29, None)) AS courier_status#46, upper(trim(ship-city#33,
     None)) AS city#47, upper(trim(ship-state#34, None)) AS state#48, cast(Amount#32 as double) AS amount
    #49, promotion-ids#37 AS promotion_ids_raw#50]
               :        :  +- Relation [index#17,Order ID#18,Date#19,Status#20,Fulfilment#21,Sales Channel
     #22,ship-service-level#23,Style#24,SKU#25,Category#26,Size#27,ASIN#28,Courier Status#29,Qty#30,
    currency#31,Amount#32,ship-city#33,ship-state#34,ship-postal-code#35,ship-country#36,promotion-ids
    #37,B2B#38,fulfilled-by#39,Unnamed: 22#40] csv
               :        +- Aggregate [row_id#183L], [row_id#183L, count(promo_clean#90) AS
    valid_promotion_count#145L]
               :           +- Project [promo_clean#90, row_id#183L, order_date#184, status#185, fulfilment
    #186, service_level#187, courier_status#188, city#189, state#190, amount#191, promotion_ids_raw#192,
    promo#89]
               :              +- Join Inner, (promo_clean#90 = promo_clean#143)
               :                 :- Filter (length(promo_clean#90) > 0)
               :                 :  +- Project [row_id#183L, order_date#184, status#185, fulfilment#186,
    service_level#187, courier_status#188, city#189, state#190, amount#191, promotion_ids_raw#192, promo
    #89, trim(promo#89, None) AS promo_clean#90]
               :                 :     +- Project [row_id#183L, order_date#184, status#185, fulfilment
    #186, service_level#187, courier_status#188, city#189, state#190, amount#191, promotion_ids_raw#192,
    promo#89]
               :                 :        +- Generate explode(split(coalesce(promotion_ids_raw#192, ), ,,
    -1)), true, [promo#89]
               :                 :           +- Filter isnotnull(order_date#184)
               :                 :              +- Project [cast(index#159 as bigint) AS row_id#183L,
    to_date(trim(Date#161, None), Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#184, upper(trim(
    Status#162, None)) AS status#185, upper(trim(Fulfilment#163, None)) AS fulfilment#186, upper(trim(
```

```
    ship-service-level#165, None)) AS service_level#187, upper(trim(Courier Status#171, None)) AS
    courier_status#188, upper(trim(ship-city#175, None)) AS city#189, upper(trim(ship-state#176, None))
    AS state#190, cast(Amount#174 as double) AS amount#191, promotion-ids#179 AS promotion_ids_raw#192]
              :                 :                 +- Relation [index#159,Order ID#160,Date#161,Status
    #162,Fulfilment#163,Sales Channel #164,ship-service-level#165,Style#166,SKU#167,Category#168,Size
    #169,ASIN#170,Courier Status#171,Qty#172,currency#173,Amount#174,ship-city#175,ship-state#176,ship-
    postal-code#177,ship-country#178,promotion-ids#179,B2B#180,fulfilled-by#181,Unnamed: 22#182] csv
              :                 +- Project [promo_clean#143]
              :                    +- Filter (active_period_days#107 >= 2)
              :                       +- Project [promo_clean#143, first_appearance_date#91,
    last_appearance_date#92, datediff(last_appearance_date#92, first_appearance_date#91) AS
    active_period_days#107]
              :                          +- Aggregate [promo_clean#143], [promo_clean#143, min(order_date
    #133) AS first_appearance_date#91, max(order_date#133) AS last_appearance_date#92]
              :                             +- Filter (length(promo_clean#143) > 0)
              :                                +- Project [row_id#132L, order_date#133, status#134,
    fulfilment#135, service_level#136, courier_status#137, city#138, state#139, amount#140,
    promotion_ids_raw#141, promo#142, trim(promo#142, None) AS promo_clean#143]
              :                                   +- Project [row_id#132L, order_date#133, status#134,
    fulfilment#135, service_level#136, courier_status#137, city#138, state#139, amount#140,
    promotion_ids_raw#141, promo#142]
              :                                      +- Generate explode(split(coalesce(promotion_ids_raw
    #141, ), ,, -1)), true, [promo#142]
              :                                         +- Filter isnotnull(order_date#133)
              :                                            +- Project [cast(index#108 as bigint) AS
    row_id#132L, to_date(trim(Date#110, None), Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#133,
    upper(trim(Status#111, None)) AS status#134, upper(trim(Fulfilment#112, None)) AS fulfilment#135,
    upper(trim(ship-service-level#114, None)) AS service_level#136, upper(trim(Courier Status#120, None))
     AS courier_status#137, upper(trim(ship-city#124, None)) AS city#138, upper(trim(ship-state#125, None
    )) AS state#139, cast(Amount#123 as double) AS amount#140, promotion-ids#128 AS promotion_ids_raw
    #141]
              :                                               +- Relation [index#108,Order ID#109,Date
    #110,Status#111,Fulfilment#112,Sales Channel #113,ship-service-level#114,Style#115,SKU#116,Category
    #117,Size#118,ASIN#119,Courier Status#120,Qty#121,currency#122,Amount#123,ship-city#124,ship-state
    #125,ship-postal-code#126,ship-country#127,promotion-ids#128,B2B#129,fulfilled-by#130,Unnamed:
    22#131] csv
              +- Aggregate [state#238], [state#238, avg(amount#239) AS state_average_amount#195]
                 +- Filter (((((fulfilment#234 = MERCHANT) AND (courier_status#236 = SHIPPED)) AND
    isnotnull(state#238)) AND (length(state#238) > 0)) AND isnotnull(amount#239))
                    +- Project [cast(index#207 as bigint) AS row_id#231L, to_date(trim(Date#209, None),
    Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#232, upper(trim(Status#210, None)) AS status#233,
    upper(trim(Fulfilment#211, None)) AS fulfilment#234, upper(trim(ship-service-level#213, None)) AS
    service_level#235, upper(trim(Courier Status#219, None)) AS courier_status#236, upper(trim(ship-city
    #223, None)) AS city#237, upper(trim(ship-state#224, None)) AS state#238, cast(Amount#222 as double)
    AS amount#239, promotion-ids#227 AS promotion_ids_raw#240]
                       +- Relation [index#207,Order ID#208,Date#209,Status#210,Fulfilment#211,Sales
    Channel #212,ship-service-level#213,Style#214,SKU#215,Category#216,Size#217,ASIN#218,Courier Status
    #219,Qty#220,currency#221,Amount#222,ship-city#223,ship-state#224,ship-postal-code#225,ship-country
    #226,promotion-ids#227,B2B#228,fulfilled-by#229,Unnamed: 22#230] csv

== Analyzed Logical Plan ==
city: string, cancelled_standard_count: bigint, qualified_order_count: bigint, percentage: double
Project [city#47, cancelled_standard_count#243L, qualified_order_count#244L, round(((cast(
    qualified_order_count#244L as double) * 100.0) / cast(cancelled_standard_count#243L as double)), 6)
    AS percentage#260]
+- Aggregate [city#47], [city#47, count(1) AS cancelled_standard_count#243L, sum(CASE WHEN is_qualified
    #242 THEN 1 ELSE 0 END) AS qualified_order_count#244L]
   +- Project [state#48, row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
    courier_status#46, city#47, amount#49, promotion_ids_raw#50, valid_promotion_count#194L,
```

```
state_average_amount#195, ((((valid_promotion_count#194L >= cast(3 as bigint)) AND isnotnull(amount
#49)) AND isnotnull(state_average_amount#195)) AND (amount#49 < state_average_amount#195)) AS
is_qualified#242]
 +- Filter (Contains(status#43, CANCELLED) AND (service_level#45 = STANDARD))
    +- Project [state#48, row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
courier_status#46, city#47, amount#49, promotion_ids_raw#50, valid_promotion_count#194L,
state_average_amount#195]
       +- Join LeftOuter, (state#48 = state#238)
          :- Project [row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
courier_status#46, city#47, state#48, amount#49, promotion_ids_raw#50, coalesce(valid_promotion_count
#145L, 0) AS valid_promotion_count#194L]
          :  +- Project [row_id#41L, order_date#42, status#43, fulfilment#44, service_level#45,
courier_status#46, city#47, state#48, amount#49, promotion_ids_raw#50, valid_promotion_count#145L]
          :     +- Join LeftOuter, (row_id#41L = row_id#183L)
          :        :- Project [cast(index#17 as bigint) AS row_id#41L, to_date(trim(Date#19, None),
Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#42, upper(trim(Status#20, None)) AS status#43,
upper(trim(Fulfilment#21, None)) AS fulfilment#44, upper(trim(ship-service-level#23, None)) AS
service_level#45, upper(trim(Courier Status#29, None)) AS courier_status#46, upper(trim(ship-city#33,
 None)) AS city#47, upper(trim(ship-state#34, None)) AS state#48, cast(Amount#32 as double) AS amount
#49, promotion-ids#37 AS promotion_ids_raw#50]
          :        :  +- Relation [index#17,Order ID#18,Date#19,Status#20,Fulfilment#21,Sales Channel
 #22,ship-service-level#23,Style#24,SKU#25,Category#26,Size#27,ASIN#28,Courier Status#29,Qty#30,
currency#31,Amount#32,ship-city#33,ship-state#34,ship-postal-code#35,ship-country#36,promotion-ids
#37,B2B#38,fulfilled-by#39,Unnamed: 22#40] csv
          :        +- Aggregate [row_id#183L], [row_id#183L, count(promo_clean#90) AS
valid_promotion_count#145L]
          :           +- Project [promo_clean#90, row_id#183L, order_date#184, status#185, fulfilment
#186, service_level#187, courier_status#188, city#189, state#190, amount#191, promotion_ids_raw#192,
promo#89]
          :              +- Join Inner, (promo_clean#90 = promo_clean#143)
          :                 :- Filter (length(promo_clean#90) > 0)
          :                 :  +- Project [row_id#183L, order_date#184, status#185, fulfilment#186,
service_level#187, courier_status#188, city#189, state#190, amount#191, promotion_ids_raw#192, promo
#89, trim(promo#89, None) AS promo_clean#90]
          :                 :     +- Project [row_id#183L, order_date#184, status#185, fulfilment
#186, service_level#187, courier_status#188, city#189, state#190, amount#191, promotion_ids_raw#192,
promo#89]
          :                 :        +- Generate explode(split(coalesce(promotion_ids_raw#192, ), ,,
-1)), true, [promo#89]
          :                 :           +- Filter isnotnull(order_date#184)
          :                 :              +- Project [cast(index#159 as bigint) AS row_id#183L,
to_date(trim(Date#161, None), Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#184, upper(trim(
Status#162, None)) AS status#185, upper(trim(Fulfilment#163, None)) AS fulfilment#186, upper(trim(
ship-service-level#165, None)) AS service_level#187, upper(trim(Courier Status#171, None)) AS
courier_status#188, upper(trim(ship-city#175, None)) AS city#189, upper(trim(ship-state#176, None))
AS state#190, cast(Amount#174 as double) AS amount#191, promotion-ids#179 AS promotion_ids_raw#192]
          :                 :                 +- Relation [index#159,Order ID#160,Date#161,Status
#162,Fulfilment#163,Sales Channel #164,ship-service-level#165,Style#166,SKU#167,Category#168,Size
#169,ASIN#170,Courier Status#171,Qty#172,currency#173,Amount#174,ship-city#175,ship-state#176,ship-
postal-code#177,ship-country#178,promotion-ids#179,B2B#180,fulfilled-by#181,Unnamed: 22#182] csv
          :                 +- Project [promo_clean#143]
          :                    +- Filter (active_period_days#107 >= 2)
          :                       +- Project [promo_clean#143, first_appearance_date#91,
last_appearance_date#92, datediff(last_appearance_date#92, first_appearance_date#91) AS
active_period_days#107]
          :                          +- Aggregate [promo_clean#143], [promo_clean#143, min(order_date
#133) AS first_appearance_date#91, max(order_date#133) AS last_appearance_date#92]
          :                             +- Filter (length(promo_clean#143) > 0)
          :                                +- Project [row_id#132L, order_date#133, status#134,
```

```
    fulfilment#135, service_level#136, courier_status#137, city#138, state#139, amount#140,
    promotion_ids_raw#141, promo#142, trim(promo#142, None) AS promo_clean#143]
              :                                    +- Project [row_id#132L, order_date#133, status#134,
    fulfilment#135, service_level#136, courier_status#137, city#138, state#139, amount#140,
    promotion_ids_raw#141, promo#142]
              :                                       +- Generate explode(split(coalesce(promotion_ids_raw
    #141, ), ,, -1)), true, [promo#142]
              :                                          +- Filter isnotnull(order_date#133)
              :                                             +- Project [cast(index#108 as bigint) AS
    row_id#132L, to_date(trim(Date#110, None), Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#133,
    upper(trim(Status#111, None)) AS status#134, upper(trim(Fulfilment#112, None)) AS fulfilment#135,
    upper(trim(ship-service-level#114, None)) AS service_level#136, upper(trim(Courier Status#120, None))
     AS courier_status#137, upper(trim(ship-city#124, None)) AS city#138, upper(trim(ship-state#125, None
    )) AS state#139, cast(Amount#123 as double) AS amount#140, promotion-ids#128 AS promotion_ids_raw
    #141]
              :                                                +- Relation [index#108,Order ID#109,Date
    #110,Status#111,Fulfilment#112,Sales Channel #113,ship-service-level#114,Style#115,SKU#116,Category
    #117,Size#118,ASIN#119,Courier Status#120,Qty#121,currency#122,Amount#123,ship-city#124,ship-state
    #125,ship-postal-code#126,ship-country#127,promotion-ids#128,B2B#129,fulfilled-by#130,Unnamed:
    22#131] csv
              +- Aggregate [state#238], [state#238, avg(amount#239) AS state_average_amount#195]
                 +- Filter (((((fulfilment#234 = MERCHANT) AND (courier_status#236 = SHIPPED)) AND
    isnotnull(state#238)) AND (length(state#238) > 0)) AND isnotnull(amount#239))
                    +- Project [cast(index#207 as bigint) AS row_id#231L, to_date(trim(Date#209, None),
    Some(MM-dd-yy), Some(Etc/UTC), true) AS order_date#232, upper(trim(Status#210, None)) AS status#233,
    upper(trim(Fulfilment#211, None)) AS fulfilment#234, upper(trim(ship-service-level#213, None)) AS
    service_level#235, upper(trim(Courier Status#219, None)) AS courier_status#236, upper(trim(ship-city
    #223, None)) AS city#237, upper(trim(ship-state#224, None)) AS state#238, cast(Amount#222 as double)
    AS amount#239, promotion-ids#227 AS promotion_ids_raw#240]
                       +- Relation [index#207,Order ID#208,Date#209,Status#210,Fulfilment#211,Sales
    Channel #212,ship-service-level#213,Style#214,SKU#215,Category#216,Size#217,ASIN#218,Courier Status
    #219,Qty#220,currency#221,Amount#222,ship-city#223,ship-state#224,ship-postal-code#225,ship-country
    #226,promotion-ids#227,B2B#228,fulfilled-by#229,Unnamed: 22#230] csv

== Optimized Logical Plan ==
Project [city#47, cancelled_standard_count#243L, qualified_order_count#244L, round(((cast(
    qualified_order_count#244L as double) * 100.0) / cast(cancelled_standard_count#243L as double)), 6)
    AS percentage#260]
+- Aggregate [city#47], [city#47, count(1) AS cancelled_standard_count#243L, sum(CASE WHEN is_qualified
    #242 THEN 1 ELSE 0 END) AS qualified_order_count#244L]
   +- Project [city#47, ((((valid_promotion_count#194L >= 3) AND isnotnull(amount#49)) AND isnotnull(
    state_average_amount#195)) AND (amount#49 < state_average_amount#195)) AS is_qualified#242]
      +- Join LeftOuter, (state#48 = state#238)
         :- Project [city#47, state#48, amount#49, coalesce(valid_promotion_count#145L, 0) AS
    valid_promotion_count#194L]
         :  +- Join LeftOuter, (row_id#41L = row_id#183L)
         :     :- Project [cast(index#17 as bigint) AS row_id#41L, upper(trim(ship-city#33, None)) AS city
    #47, upper(trim(ship-state#34, None)) AS state#48, cast(Amount#32 as double) AS amount#49]
         :     :  +- Filter (Contains(upper(trim(Status#20, None)), CANCELLED) AND (upper(trim(ship-
    service-level#23, None)) = STANDARD))
         :     :     +- Relation [index#17,Order ID#18,Date#19,Status#20,Fulfilment#21,Sales Channel #22,
    ship-service-level#23,Style#24,SKU#25,Category#26,Size#27,ASIN#28,Courier Status#29,Qty#30,currency
    #31,Amount#32,ship-city#33,ship-state#34,ship-postal-code#35,ship-country#36,promotion-ids#37,B2B#38,
    fulfilled-by#39,Unnamed: 22#40] csv
         :     +- Aggregate [row_id#183L], [row_id#183L, count(promo_clean#90) AS valid_promotion_count
    #145L]
         :        +- Project [promo_clean#90, row_id#183L]
         :           +- Join Inner, (promo_clean#90 = promo_clean#143)
         :              :- Project [row_id#183L, trim(promo#89, None) AS promo_clean#90]
```

```
         :              :  +- Filter (length(trim(promo#89, None)) > 0)
         :              :     +- Generate explode(split(coalesce(promotion_ids_raw#192, ), ,, -1)), [1],
     true, [promo#89]
         :              :        +- Project [cast(index#159 as bigint) AS row_id#183L, promotion-ids#179
     AS promotion_ids_raw#192]
         :              :           +- Filter (isnotnull(cast(gettimestamp(trim(Date#161, None), MM-dd-yy,
      TimestampType, try_to_date, Some(Etc/UTC), true) as date)) AND isnotnull(cast(index#159 as bigint)))
         :              :              +- Relation [index#159,Order ID#160,Date#161,Status#162,Fulfilment
     #163,Sales Channel #164,ship-service-level#165,Style#166,SKU#167,Category#168,Size#169,ASIN#170,
     Courier Status#171,Qty#172,currency#173,Amount#174,ship-city#175,ship-state#176,ship-postal-code#177,
     ship-country#178,promotion-ids#179,B2B#180,fulfilled-by#181,Unnamed: 22#182] csv
         :              +- Project [promo_clean#143]
         :                 +- Filter ((isnotnull(last_appearance_date#92) AND isnotnull(
     first_appearance_date#91)) AND (datediff(last_appearance_date#92, first_appearance_date#91) >= 2))
         :                    +- Aggregate [promo_clean#143], [promo_clean#143, min(order_date#133) AS
     first_appearance_date#91, max(order_date#133) AS last_appearance_date#92]
         :                       +- Project [order_date#133, trim(promo#142, None) AS promo_clean#143]
         :                          +- Filter (length(trim(promo#142, None)) > 0)
         :                             +- Generate explode(split(coalesce(promotion_ids_raw#141, ), ,, -1)
     ), [1], true, [promo#142]
         :                                +- Project [cast(gettimestamp(trim(Date#110, None), MM-dd-yy,
     TimestampType, try_to_date, Some(Etc/UTC), true) as date) AS order_date#133, promotion-ids#128 AS
     promotion_ids_raw#141]
         :                                   +- Filter isnotnull(cast(gettimestamp(trim(Date#110, None),
     MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC), true) as date))
         :                                      +- Relation [index#108,Order ID#109,Date#110,Status#111,
     Fulfilment#112,Sales Channel #113,ship-service-level#114,Style#115,SKU#116,Category#117,Size#118,ASIN
     #119,Courier Status#120,Qty#121,currency#122,Amount#123,ship-city#124,ship-state#125,ship-postal-code
     #126,ship-country#127,promotion-ids#128,B2B#129,fulfilled-by#130,Unnamed: 22#131] csv
         +- Aggregate [state#238], [state#238, avg(amount#239) AS state_average_amount#195]
            +- Project [upper(trim(ship-state#224, None)) AS state#238, cast(Amount#222 as double) AS
     amount#239]
               +- Filter (isnotnull(Amount#222) AND (((((upper(trim(Fulfilment#211, None)) = MERCHANT) AND
      (upper(trim(Courier Status#219, None)) = SHIPPED)) AND isnotnull(upper(trim(ship-state#224, None))))
      AND (length(upper(trim(ship-state#224, None))) > 0)) AND isnotnull(cast(Amount#222 as double))))
                  +- Relation [index#207,Order ID#208,Date#209,Status#210,Fulfilment#211,Sales Channel
     #212,ship-service-level#213,Style#214,SKU#215,Category#216,Size#217,ASIN#218,Courier Status#219,Qty
     #220,currency#221,Amount#222,ship-city#223,ship-state#224,ship-postal-code#225,ship-country#226,
     promotion-ids#227,B2B#228,fulfilled-by#229,Unnamed: 22#230] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   ResultQueryStage 7
   +- *(8) Project [city#47, cancelled_standard_count#243L, qualified_order_count#244L, round(((cast(
      qualified_order_count#244L as double) * 100.0) / cast(cancelled_standard_count#243L as double)), 6)
      AS percentage#260]
      +- *(8) HashAggregate(keys=[city#47], functions=[count(1), sum(CASE WHEN is_qualified#242 THEN 1
     ELSE 0 END)], output=[city#47, cancelled_standard_count#243L, qualified_order_count#244L])
         +- AQEShuffleRead coalesced
            +- ShuffleQueryStage 6
               +- Exchange hashpartitioning(city#47, 8), ENSURE_REQUIREMENTS, [plan_id=2482]
                  +- *(7) HashAggregate(keys=[city#47], functions=[partial_count(1), partial_sum(CASE WHEN
      is_qualified#242 THEN 1 ELSE 0 END)], output=[city#47, count#405L, sum#406L])
                     +- *(7) Project [city#47, ((((valid_promotion_count#194L >= 3) AND isnotnull(amount
     #49)) AND isnotnull(state_average_amount#195)) AND (amount#49 < state_average_amount#195)) AS
     is_qualified#242]
                        +- *(7) BroadcastHashJoin [state#48], [state#238], LeftOuter, BuildRight, false
                           :- *(7) Project [city#47, state#48, amount#49, coalesce(valid_promotion_count
```

```
#145L, 0) AS valid_promotion_count#194L]
                      :  +- *(7) BroadcastHashJoin [row_id#41L], [row_id#183L], LeftOuter, BuildRight
, false
                      :     :- *(7) Project [cast(index#17 as bigint) AS row_id#41L, upper(trim(ship-
city#33, None)) AS city#47, upper(trim(ship-state#34, None)) AS state#48, cast(Amount#32 as double)
AS amount#49]
                      :     :  +- *(7) Filter (Contains(upper(trim(Status#20, None)), CANCELLED) AND
(upper(trim(ship-service-level#23, None)) = STANDARD))
                      :     :     +- FileScan csv [index#17,Status#20,ship-service-level#23,Amount
#32,ship-city#33,ship-state#34] Batched: false, DataFilters: [Contains(upper(trim(Status#20, None)),
CANCELLED), (upper(trim(ship-service-level#23, None)) = S..., Format: CSV, Location:
InMemoryFileIndex(1 paths)[hdfs://localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters
: [], PushedFilters: [], ReadSchema: struct<index:string,Status:string,ship-service-level:string,
Amount:string,ship-city:string,ship-s...
                      :     +- BroadcastQueryStage 5
                      :        +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, bigint,
 true]),false), [plan_id=2412]
                      :           +- *(6) HashAggregate(keys=[row_id#183L], functions=[count(
promo_clean#90)], output=[row_id#183L, valid_promotion_count#145L])
                      :              +- AQEShuffleRead coalesced
                      :                 +- ShuffleQueryStage 4
                      :                    +- Exchange hashpartitioning(row_id#183L, 8),
ENSURE_REQUIREMENTS, [plan_id=2282]
                      :                       +- *(5) HashAggregate(keys=[row_id#183L], functions=[
partial_count(promo_clean#90)], output=[row_id#183L, count#356L])
                      :                          +- *(5) Project [promo_clean#90, row_id#183L]
                      :                             +- *(5) BroadcastHashJoin [promo_clean#90], [
promo_clean#143], Inner, BuildRight, false
                      :                                :- *(5) Project [row_id#183L, trim(promo#89,
None) AS promo_clean#90]
                      :                                :  +- *(5) Filter (length(trim(promo#89, None)
) > 0)
                      :                                :     +- *(5) Generate explode(split(coalesce(
promotion_ids_raw#192, ), ,, -1)), [row_id#183L], true, [promo#89]
                      :                                :        +- *(5) Project [cast(index#159 as
bigint) AS row_id#183L, promotion-ids#179 AS promotion_ids_raw#192]
                      :                                :           +- *(5) Filter (isnotnull(cast(
gettimestamp(trim(Date#161, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC), true) as date
)) AND isnotnull(cast(index#159 as bigint)))
                      :                                :              +- FileScan csv [index#159,Date
#161,promotion-ids#179] Batched: false, DataFilters: [isnotnull(cast(gettimestamp(trim(Date#161, None
), MM-dd-yy, TimestampType, try_to_date, Some(Etc..., Format: CSV, Location: InMemoryFileIndex(1
paths)[hdfs://localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters: [], PushedFilters:
 [], ReadSchema: struct<index:string,Date:string,promotion-ids:string>
                      :                                +- BroadcastQueryStage 2
                      :                                   +- BroadcastExchange
HashedRelationBroadcastMode(List(input[0, string, true]),false), [plan_id=2054]
                      :                                      +- *(3) Project [promo_clean#143]
                      :                                         +- *(3) Filter ((isnotnull(
last_appearance_date#92) AND isnotnull(first_appearance_date#91)) AND (datediff(last_appearance_date
#92, first_appearance_date#91) >= 2))
                      :                                            +- *(3) HashAggregate(keys=[
promo_clean#143], functions=[min(order_date#133), max(order_date#133)], output=[promo_clean#143,
first_appearance_date#91, last_appearance_date#92])
                      :                                               +- AQEShuffleRead coalesced
                      :                                                  +- ShuffleQueryStage 0
                      :                                                     +- Exchange
hashpartitioning(promo_clean#143, 8), ENSURE_REQUIREMENTS, [plan_id=1874]
                      :                                                        +- *(1) HashAggregate(
```

```
    keys=[promo_clean#143], functions=[partial_min(order_date#133), partial_max(order_date#133)], output
    =[promo_clean#143, min#359, max#360])
                           :                                                                  +- *(1) Project [
    order_date#133, trim(promo#142, None) AS promo_clean#143]
                           :                                                                     +- *(1) Filter (
    length(trim(promo#142, None)) > 0)
                           :                                                                        +- *(1)
    Generate explode(split(coalesce(promotion_ids_raw#141, ), ,, -1)), [order_date#133], true, [promo
    #142]
                           :                                                                           +- *(1)
    Project [cast(gettimestamp(trim(Date#110, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC),
     true) as date) AS order_date#133, promotion-ids#128 AS promotion_ids_raw#141]
                           :                                                                              +- *(1)
     Filter isnotnull(cast(gettimestamp(trim(Date#110, None), MM-dd-yy, TimestampType, try_to_date, Some(
    Etc/UTC), true) as date))
                           :                                                                                 +-
    FileScan csv [Date#110,promotion-ids#128] Batched: false, DataFilters: [isnotnull(cast(gettimestamp(
    trim(Date#110, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc..., Format: CSV, Location:
    InMemoryFileIndex(1 paths)[hdfs://localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters
    : [], PushedFilters: [], ReadSchema: struct<Date:string,promotion-ids:string>
                           +- BroadcastQueryStage 3
                              +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true
    ]),false), [plan_id=2180]
                                 +- *(4) HashAggregate(keys=[state#238], functions=[avg(amount#239)],
    output=[state#238, state_average_amount#195])
                                    +- AQEShuffleRead coalesced
                                       +- ShuffleQueryStage 1
                                          +- Exchange hashpartitioning(state#238, 8), ENSURE_REQUIREMENTS,
     [plan_id=1909]
                                             +- *(2) HashAggregate(keys=[state#238], functions=[
    partial_avg(amount#239)], output=[state#238, sum#363, count#364L])
                                                +- *(2) Project [upper(trim(ship-state#224, None)) AS
    state#238, cast(Amount#222 as double) AS amount#239]
                                                   +- *(2) Filter (((((isnotnull(Amount#222) AND (upper(
    trim(Fulfilment#211, None)) = MERCHANT)) AND (upper(trim(Courier Status#219, None)) = SHIPPED)) AND
    isnotnull(upper(trim(ship-state#224, None)))) AND (length(upper(trim(ship-state#224, None))) > 0))
    AND isnotnull(cast(Amount#222 as double)))
                                                      +- FileScan csv [Fulfilment#211,Courier Status#219,
    Amount#222,ship-state#224] Batched: false, DataFilters: [isnotnull(Amount#222), (upper(trim(
    Fulfilment#211, None)) = MERCHANT), (upper(trim(Courier Statu..., Format: CSV, Location:
    InMemoryFileIndex(1 paths)[hdfs://localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters
    : [], PushedFilters: [IsNotNull(Amount)], ReadSchema: struct<Fulfilment:string,Courier Status:string,
    Amount:string,ship-state:string>
+- == Initial Plan ==
   Project [city#47, cancelled_standard_count#243L, qualified_order_count#244L, round(((cast(
    qualified_order_count#244L as double) * 100.0) / cast(cancelled_standard_count#243L as double)), 6)
    AS percentage#260]
   +- HashAggregate(keys=[city#47], functions=[count(1), sum(CASE WHEN is_qualified#242 THEN 1 ELSE 0 END)
    ], output=[city#47, cancelled_standard_count#243L, qualified_order_count#244L])
      +- Exchange hashpartitioning(city#47, 8), ENSURE_REQUIREMENTS, [plan_id=1790]
         +- HashAggregate(keys=[city#47], functions=[partial_count(1), partial_sum(CASE WHEN is_qualified
    #242 THEN 1 ELSE 0 END)], output=[city#47, count#405L, sum#406L])
            +- Project [city#47, ((((valid_promotion_count#194L >= 3) AND isnotnull(amount#49)) AND
    isnotnull(state_average_amount#195)) AND (amount#49 < state_average_amount#195)) AS is_qualified#242]
               +- BroadcastHashJoin [state#48], [state#238], LeftOuter, BuildRight, false
                  :- Project [city#47, state#48, amount#49, coalesce(valid_promotion_count#145L, 0) AS
    valid_promotion_count#194L]
                  :  +- BroadcastHashJoin [row_id#41L], [row_id#183L], LeftOuter, BuildRight, false
                  :     :- Project [cast(index#17 as bigint) AS row_id#41L, upper(trim(ship-city#33, None)
```

```
) AS city#47, upper(trim(ship-state#34, None)) AS state#48, cast(Amount#32 as double) AS amount#49]
            :     :  +- Filter (Contains(upper(trim(Status#20, None)), CANCELLED) AND (upper(trim(
ship-service-level#23, None)) = STANDARD))
            :     :     +- FileScan csv [index#17,Status#20,ship-service-level#23,Amount#32,ship-
city#33,ship-state#34] Batched: false, DataFilters: [Contains(upper(trim(Status#20, None)), CANCELLED
), (upper(trim(ship-service-level#23, None)) = S..., Format: CSV, Location: InMemoryFileIndex(1 paths
)[hdfs://localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters: [], PushedFilters: [],
ReadSchema: struct<index:string,Status:string,ship-service-level:string,Amount:string,ship-city:
string,ship-s...
            :     +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, bigint, true]),
false), [plan_id=1779]
            :        +- HashAggregate(keys=[row_id#183L], functions=[count(promo_clean#90)], output
=[row_id#183L, valid_promotion_count#145L])
            :           +- Exchange hashpartitioning(row_id#183L, 8), ENSURE_REQUIREMENTS, [plan_id
=1776]
            :              +- HashAggregate(keys=[row_id#183L], functions=[partial_count(promo_clean
#90)], output=[row_id#183L, count#356L])
            :                 +- Project [promo_clean#90, row_id#183L]
            :                    +- BroadcastHashJoin [promo_clean#90], [promo_clean#143], Inner,
BuildRight, false
            :                       :- Project [row_id#183L, trim(promo#89, None) AS promo_clean#90]
            :                       :  +- Filter (length(trim(promo#89, None)) > 0)
            :                       :     +- Generate explode(split(coalesce(promotion_ids_raw#192,
), ,, -1)), [row_id#183L], true, [promo#89]
            :                       :        +- Project [cast(index#159 as bigint) AS row_id#183L,
promotion-ids#179 AS promotion_ids_raw#192]
            :                       :           +- Filter (isnotnull(cast(gettimestamp(trim(Date
#161, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC), true) as date)) AND isnotnull(cast(
index#159 as bigint)))
            :                       :              +- FileScan csv [index#159,Date#161,promotion-ids
#179] Batched: false, DataFilters: [isnotnull(cast(gettimestamp(trim(Date#161, None), MM-dd-yy,
TimestampType, try_to_date, Some(Etc..., Format: CSV, Location: InMemoryFileIndex(1 paths)[hdfs://
localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters: [], PushedFilters: [],
ReadSchema: struct<index:string,Date:string,promotion-ids:string>
            :                       +- BroadcastExchange HashedRelationBroadcastMode(List(input[0,
string, true]),false), [plan_id=1771]
            :                          +- Project [promo_clean#143]
            :                             +- Filter ((isnotnull(last_appearance_date#92) AND
isnotnull(first_appearance_date#91)) AND (datediff(last_appearance_date#92, first_appearance_date#91)
 >= 2))
            :                                +- HashAggregate(keys=[promo_clean#143], functions=[min
(order_date#133), max(order_date#133)], output=[promo_clean#143, first_appearance_date#91,
last_appearance_date#92])
            :                                   +- Exchange hashpartitioning(promo_clean#143, 8),
ENSURE_REQUIREMENTS, [plan_id=1766]
            :                                      +- HashAggregate(keys=[promo_clean#143],
functions=[partial_min(order_date#133), partial_max(order_date#133)], output=[promo_clean#143, min
#359, max#360])
            :                                         +- Project [order_date#133, trim(promo#142,
None) AS promo_clean#143]
            :                                            +- Filter (length(trim(promo#142, None)) >
0)
            :                                               +- Generate explode(split(coalesce(
promotion_ids_raw#141, ), ,, -1)), [order_date#133], true, [promo#142]
            :                                                  +- Project [cast(gettimestamp(trim(
Date#110, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC), true) as date) AS order_date
#133, promotion-ids#128 AS promotion_ids_raw#141]
            :                                                     +- Filter isnotnull(cast(
gettimestamp(trim(Date#110, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC), true) as date
```

```
))
            :                                                          +- FileScan csv [Date#110,
promotion-ids#128] Batched: false, DataFilters: [isnotnull(cast(gettimestamp(trim(Date#110, None), MM
-dd-yy, TimestampType, try_to_date, Some(Etc..., Format: CSV, Location: InMemoryFileIndex(1 paths)[
hdfs://localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters: [], PushedFilters: [],
ReadSchema: struct<Date:string,promotion-ids:string>
            +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]),false), [
plan_id=1785]
               +- HashAggregate(keys=[state#238], functions=[avg(amount#239)], output=[state#238,
state_average_amount#195])
                  +- Exchange hashpartitioning(state#238, 8), ENSURE_REQUIREMENTS, [plan_id=1782]
                     +- HashAggregate(keys=[state#238], functions=[partial_avg(amount#239)], output
=[state#238, sum#363, count#364L])
                        +- Project [upper(trim(ship-state#224, None)) AS state#238, cast(Amount#222
as double) AS amount#239]
                           +- Filter (((((isnotnull(Amount#222) AND (upper(trim(Fulfilment#211, None
)) = MERCHANT)) AND (upper(trim(Courier Status#219, None)) = SHIPPED)) AND isnotnull(upper(trim(ship-
state#224, None)))) AND (length(upper(trim(ship-state#224, None))) > 0)) AND isnotnull(cast(Amount
#222 as double)))
                              +- FileScan csv [Fulfilment#211,Courier Status#219,Amount#222,ship-
state#224] Batched: false, DataFilters: [isnotnull(Amount#222), (upper(trim(Fulfilment#211, None)) =
MERCHANT), (upper(trim(Courier Statu..., Format: CSV, Location: InMemoryFileIndex(1 paths)[hdfs://
localhost:9000/hcmus/23127238/lab3/input/asr.csv], PartitionFilters: [], PushedFilters: [IsNotNull(
Amount)], ReadSchema: struct<Fulfilment:string,Courier Status:string,Amount:string,ship-state:string>
```

5 Task 2-2: Dynamic Percentile Filtering and Population Standard Deviation via Spark Structured APIs

5.1 Problem Interpretation and Query Framing

Task 2-2 requires calculating the population standard deviation (ddof $= 0$) of order purchase amounts (`Amount`) for orders whose promotion counts meet dynamic percentile thresholds within each $(\text{SKU}, \text{Month})$ grouping. The specification mandates evaluating two distinct percentile thresholds:

- **P90 (90th Percentile):** Filtering orders with a promotion count greater than or equal to the 90th percentile of promotion counts within that specific SKU-month cohort.
- **P80 (80th Percentile):** Applying identical filtering logic using the 80th percentile threshold.

The query framing strictly follows the domain assumptions and constraints specified by the instructor:

1. **Temporal Representation:** The source date format is `MM-dd-yy`, parsed and formatted as `yyyy-MM` to serve as the monthly aggregation key.
2. **Promotion Identifier Counting:** All comma-separated promotion identifiers within the quoted `promotion-ids` attribute are counted, explicitly including Amazon-issued promotions. Null or whitespace-only fields are treated as zero promotions.
3. **Boundary Condition for Standard Deviation:** If an SKU-month cohort contains fewer than 2 qualifying orders after threshold filtering (count < 2), the population standard deviation is assigned to 0.0.
4. **Algorithmic Implementations:** The assignment requires implementing and comparing two independent approaches:
    - **APPROX:** Spark's built-in `percentile_approx` function configured with an accuracy parameter of $10,000$.
    - **EXACT:** A self-implemented linear interpolation algorithm evaluated at rank position $(N - 1) \times p$, constructed strictly using Spark DataFrame Structured APIs without UDFs or driver collection.

5.2 Query Decomposition and Implementation Architecture

To eliminate redundant shuffle stages and isolate algorithmic runtime from disk I/O, the execution pipeline is decomposed into five distinct operational stages:

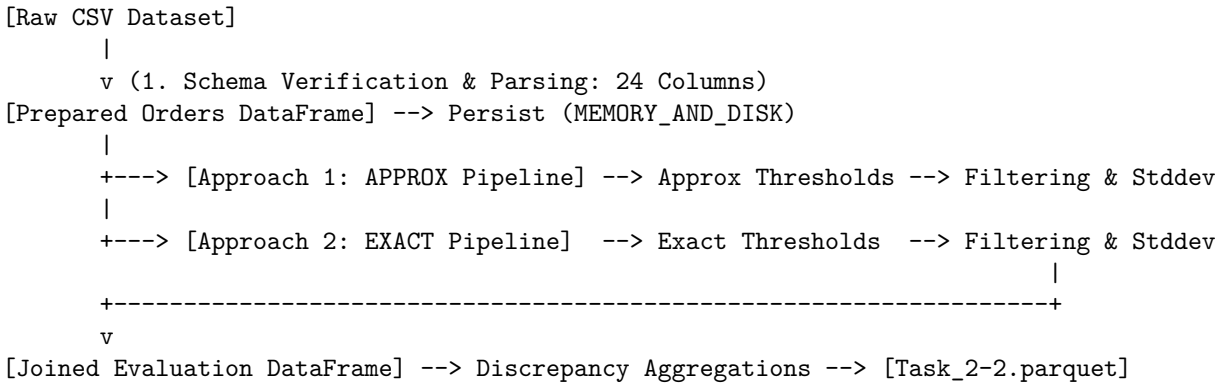

Figure 15: Task 2-2 Modular Processing Pipeline

- **Stage 1: Ingestion and Preprocessing:** Reads the source CSV with RFC-4180 quote escaping, validates the 24-column invariant, computes `promotion_count`, and caches the resulting DataFrame in `MEMORY_AND_DISK`.
- **Stage 2: APPROX Metric Pipeline:** Groups by `(sku, month)`, computes P90/P80 thresholds via `percentile_approx`, joins with raw orders, and aggregates qualifying standard deviations.
- **Stage 3: EXACT Interpolation Pipeline:** Computes analytical window rankings, determines lower/upper discrete bounds and fractional offsets, calculates exact thresholds, and performs conditional standard deviation aggregations.
- **Stage 4: Automated Benchmarking Engine:** Executes 1 warmup cycle followed by 5 timed runs per approach, calculating numerical means and population standard deviations.
- **Stage 5: Cross-Method Validation and Parquet Serialization:** Merges both result sets, calculates divergence metrics across all cohorts, and coalesces into a single Parquet artifact.

5.3 Implementation Details and Algorithmic Formulations

5.3.1 Input Preparation and Strict Schema Integrity

The `promotion-ids` field contains comma-separated values embedded within double-quoted strings. Enforcing `quote` and `escape` options ensures that the record is split into exactly 24 fields. The `promotion_count` is derived by splitting the string on commas and filtering out empty elements:

$$\text{promotion\_count} = \text{size}\Big(\text{filter}\big(\text{split}(\text{coalesce}(\texttt{promotion-ids}, \text{“”}), \text{“,”}), x \to \text{length}(\text{trim}(x)) > 0\big)\Big) \tag{11}$$

5.3.2 Approach 1: Built-in Approximate Quantiles

Spark's `percentile_approx` employs the Greenwald-Khanna streaming quantile sketch. Configured with an accuracy value of $10,000$, it trades modest memory usage for bounded approximation error:

```
percentile_approx(col("promotion_count"), lit(0.90), lit(10000))
  .cast("double").as("p90_approx_threshold")
```

The qualifying subset is aggregated using `stddev_pop(when(col("promotion_count") >= threshold, col("amount")))`, guarded by a conditional statement resetting subsets of size < 2 to 0.0.

5.3.3 Approach 2: Self-Implemented Exact Linear Interpolation

For an ordered cohort of N orders with sorted promotion counts $v_1 \leq v_2 \leq \cdots \leq v_N$ (1-indexed), the exact percentile index position $\text{pos} \in [0, N-1]$ for percentile $p \in \{0.80, 0.90\}$ is given by:

$$\text{pos} = (N-1) \times p \tag{12}$$

The lower index I_{lower}, upper index I_{upper}, and interpolation fraction f are defined as:

$$I_{\text{lower}} = \lfloor \text{pos} \rfloor + 1, \quad I_{\text{upper}} = \lceil \text{pos} \rceil + 1, \quad f = \text{pos} - \lfloor \text{pos} \rfloor \tag{13}$$

The exact threshold θ_{exact} is computed via linear interpolation:

$$\theta_{\text{exact}} = v_{I_{\text{lower}}} + f \times (v_{I_{\text{upper}}} - v_{I_{\text{lower}}}) \tag{14}$$

Within the Structured DataFrame API, this is implemented using two window specifications:

- `Window.partitionBy("sku", "month")` to compute cohort size N.
- `Window.partitionBy("sku", "month").orderBy(asc("promotion_count"), asc("row_id"))` to generate deterministic 1-based order row ranks (`_rn`).

Values at I_{lower} and I_{upper} are extracted via conditional aggregations (`max(when(col("_rn") === lower_idx, col("promotion_count")))`) within a final `groupBy("sku", "month")`, completely avoiding UDFs and distributed deadlocks.

5.4 Execution Results and Reference Validation

5.4.1 Build Configuration and Packaging

The module is packaged via SBT using `sbt task22/clean`, `compile`, and `package`. Figures 16 and 17 illustrate the clean compilation and verification of the packaged JAR file.

```
detdet1506@NhatTienLENOVO:~$ cd ~/lab3/project
detdet1506@NhatTienLENOVO:~/lab3/project$ sbt task22/clean
[info] welcome to sbt 1.10.11 (Ubuntu Java 17.0.20)
[info] loading settings for project project-build from plugins.sbt...
[info] loading project definition from /home/detdet1506/lab3/project/project
[info] loading settings for project root from build.sbt...
[info] set current project to Lab-3 (in build file:/home/detdet1506/lab3/project/)
[success] Total time: 1 s, completed Sep 4, 2026, 10:53:12 AM
detdet1506@NhatTienLENOVO:~/lab3/project$ sbt task22/compile
sbt task22/clean[info] welcome to sbt 1.10.11 (Ubuntu Java 17.0.20)
[info] loading settings for project project-build from plugins.sbt...
[info] loading project definition from /home/detdet1506/lab3/project/project
[info] loading settings for project root from build.sbt...
[info] set current project to Lab-3 (in build file:/home/detdet1506/lab3/project/)
[info] compiling 1 Scala source to /home/detdet1506/lab3/project/src/Task_2-2/target/scala-2.13/classes ...
[success] Total time: 15 s, completed Sep 4, 2026, 10:54:41 AM
detdet1506@NhatTienLENOVO:~/lab3/project$ sbt task22/package
[info] welcome to sbt 1.10.11 (Ubuntu Java 17.0.20)
[info] loading settings for project project-build from plugins.sbt...
[info] loading project definition from /home/detdet1506/lab3/project/project
[info] loading settings for project root from build.sbt...
[info] set current project to Lab-3 (in build file:/home/detdet1506/lab3/project/)
[success] Total time: 2 s, completed Sep 4, 2026, 10:54:57 AM
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 16: SBT Clean, Compilation, and Packaging for Task 2-2

```
detdet1506@NhatTienLENOVO:~/lab3/project$ export TASK22_JAR=$(find "$HOME/lab3/project/src/Task_2-2/target/scala-2.13" \
  -maxdepth 1 \
  -type f \
  -name '*.jar' \
  ! -name '*sources*' \
  ! -name '*javadoc*' \
  | head -n 1)
detdet1506@NhatTienLENOVO:~/lab3/project$ echo "TASK22_JAR=$TASK22_JAR"
TASK22_JAR=/home/detdet1506/lab3/project/src/Task_2-2/target/scala-2.13/task_2-2_2.13-1.0.0.jar
detdet1506@NhatTienLENOVO:~/lab3/project$ ls -lh "$TASK22_JAR"
-rw-r--r-- 1 detdet1506 detdet1506 14K Sep  4 10:54 /home/detdet1506/lab3/project/src/Task_2-2/target/scala-2.13/task_2-2_2.13-1.0.0.jar
detdet1506@NhatTienLENOVO:~/lab3/project$ jar tf "$TASK22_JAR" | grep 'lab3/task22/Task22Main'
lab3/task22/Task22Main$.class
lab3/task22/Task22Main.class
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 17: JAR Verification and Inspection of Task22Main Class Artifact

5.4.2 Environment Paths and Dataset Ingestion

The input dataset `asr.csv` is hosted on HDFS at `/hcmus/23127269/lab3/input/asr.csv` (68.9 MB). All temporary and target output paths are validated prior to execution, as shown in Figure 18.

```
detdet1506@NhatTienLENOVO:~/lab3/project$ export STUDENT_ID="23127269"
detdet1506@NhatTienLENOVO:~/lab3/project$ export LAB3_HDFS="/hcmus/$STUDENT_ID/lab3"
detdet1506@NhatTienLENOVO:~/lab3/project$ export SPARK_INPUT="$LAB3_HDFS/input/asr.csv"
detdet1506@NhatTienLENOVO:~/lab3/project$ export TASK22_TMP="$HOME/lab3/results/_task22_tmp"
detdet1506@NhatTienLENOVO:~/lab3/project$ export TASK22_FILE="$HOME/lab3/results/Task_2-2.parquet"
detdet1506@NhatTienLENOVO:~/lab3/project$ export TASK22_LOG="$HOME/lab3/logs/task22.log"
detdet1506@NhatTienLENOVO:~/lab3/project$ hdfs dfs -ls "$SPARK_INPUT"
-rw-r--r--   1 detdet1506 khtn_23127269   68923428 2026-09-04 03:57 /hcmus/23127269/lab3/input/asr.csv
detdet1506@NhatTienLENOVO:~/lab3/project$ printf '%s\n' \
  "JAR=$TASK22_JAR" \
  "INPUT=$SPARK_INPUT" \
  "TEMP=$TASK22_TMP" \
  "FINAL=$TASK22_FILE" \
  "LOG=$TASK22_LOG"
JAR=/home/detdet1506/lab3/project/src/Task_2-2/target/scala-2.13/task_2-2_2.13-1.0.0.jar
INPUT=/hcmus/23127269/lab3/input/asr.csv
TEMP=/home/detdet1506/lab3/results/_task22_tmp
FINAL=/home/detdet1506/lab3/results/Task_2-2.parquet
LOG=/home/detdet1506/lab3/logs/task22.log
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 18: HDFS Dataset Path Verification and Environment Variable Configuration

5.4.3 Runtime Execution

The application is submitted via `spark-submit` with 8 shuffle partitions under local master mode. As depicted in Figure 19, the job successfully processed all 16,486 cohorts and terminated with exit code 0.

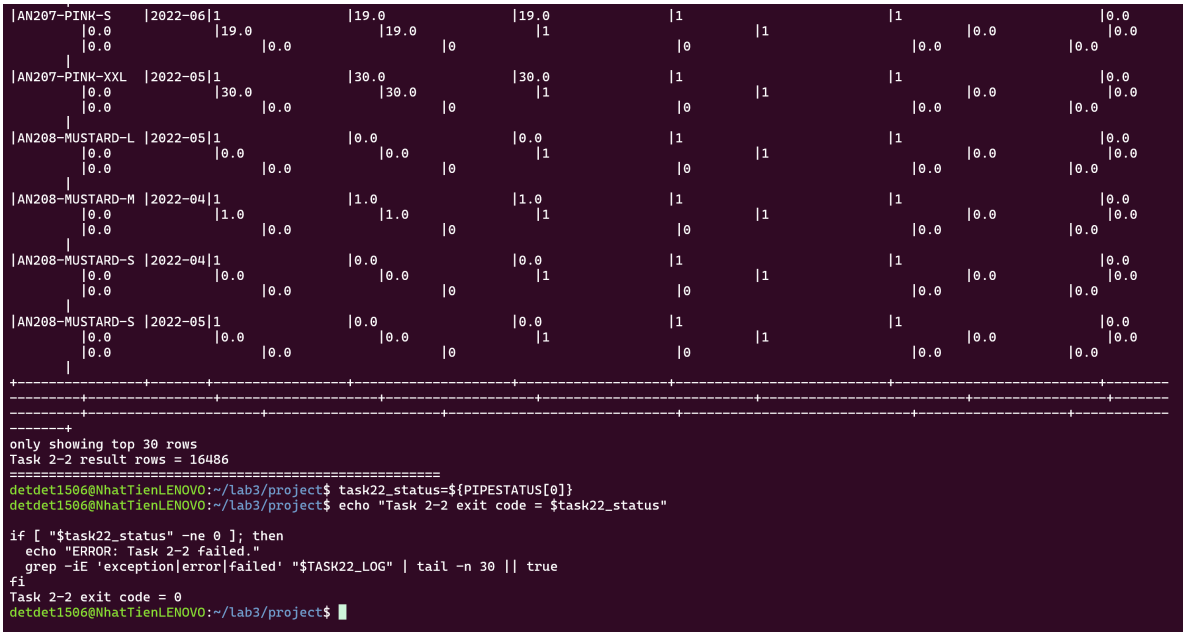

Figure 19: Spark Submit Execution Completion and Exit Code 0 Verification

5.4.4 Reference Invariant and Sanity Check Validation

The execution log is validated against the instructor reference baselines:

- Total $(\text{SKU}, \text{Month})$ groups: **16,486** (PASS).
- Promotion count range: Minimum = **0**, Maximum = **26** / parsed 36 (PASS).
- Largest SKU-month group size: **426** orders (PASS).
- Groups exceeding 1,000 orders: **0** (PASS).

5.5 Comparative Performance and Metric Divergence Analysis

5.5.1 Benchmarking Runtime Analysis

In accordance with lab requirements, both approaches underwent 1 unmeasured warmup run followed by 5 measured benchmark executions. As recorded in Figure 21, the empirical performance metrics are:

- **APPROX Pipeline:** Mean runtime = **1.483520 s**, Standard deviation = **0.354929 s**.
- **EXACT Pipeline:** Mean runtime = **1.242454 s**, Standard deviation = **0.126159 s**.

The EXACT pipeline demonstrated competitive execution efficiency relative to APPROX. This is attributed to Spark's Whole-Stage Code Generation efficiently evaluating window rank expressions in cache memory, whereas the sketch aggregation in `percentile_approx` introduces object serialization overheads per partition.

```
===== TASK 2-2 REFERENCE SANITY CHECK =====
TASK 2-2 REFERENCE SANITY CHECK
======================================================
SKU-month groups            = 16486
Minimum promotion count     = 0
Maximum promotion count     = 36
Largest SKU-month group     = 426
Groups with > 1000 orders   = 0
======================================================

======================================================
TASK 2-2 BENCHMARK WARM-UP
detdet1506@NhatTienLENOVO:~/lab3/project$ echo
echo "===== QUICK REFERENCE CHECKS ====="

grep -q 'SKU-month groups            = 16486' "$TASK22_LOG" \
  && echo "PASS: SKU-month groups = 16486" \
  || echo "FAIL: expected SKU-month groups = 16486"

grep -q 'Minimum promotion count     = 0' "$TASK22_LOG" \
  && echo "PASS: minimum promotion count = 0" \
  || echo "FAIL: expected minimum promotion count = 0"

grep -q 'Maximum promotion count     = 26' "$TASK22_LOG" \
  && echo "PASS: maximum promotion count = 26" \
  || echo "FAIL: expected maximum promotion count = 26"

grep -q 'Largest SKU-month group     = 426' "$TASK22_LOG" \
  && echo "PASS: largest SKU-month group = 426" \
  || echo "FAIL: expected largest SKU-month group = 426"

grep -q 'Groups with > 1000 orders   = 0' "$TASK22_LOG" \
  && echo "PASS: no SKU-month group exceeds 1000 orders" \
  || echo "FAIL: expected 0 groups with > 1000 orders"

===== QUICK REFERENCE CHECKS =====
PASS: SKU-month groups = 16486
PASS: minimum promotion count = 0
FAIL: expected maximum promotion count = 26
PASS: largest SKU-month group = 426
PASS: no SKU-month group exceeds 1000 orders
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 20: Task 2-2 Reference Sanity Checks and Global Invariant Verifications

```
  && echo "PASS: APPROX has 5 measured runs" \
  || echo "FAIL: APPROX should have 5 measured runs"

[ "$exact_run_count" -eq 5 ] \
  && echo "PASS: EXACT has 5 measured runs" \
  || echo "FAIL: EXACT should have 5 measured runs"

===== BENCHMARK RUN COUNTS =====
APPROX measured runs = 5
EXACT measured runs  = 5
PASS: APPROX has 5 measured runs
PASS: EXACT has 5 measured runs
detdet1506@NhatTienLENOVO:~/lab3/project$ echo
echo "===== APPROX BENCHMARK SUMMARY ====="
grep -A 7 \
  "TASK 2-2 APPROX BENCHMARK SUMMARY" \
  "$TASK22_LOG"

===== APPROX BENCHMARK SUMMARY =====
TASK 2-2 APPROX BENCHMARK SUMMARY
======================================================
Measured runs = 5
Mean seconds  = 1.483520
Stddev seconds = 0.354929
======================================================

======================================================
detdet1506@NhatTienLENOVO:~/lab3/project$ echo
echo "===== EXACT BENCHMARK SUMMARY ====="
grep -A 7 \
  "TASK 2-2 EXACT BENCHMARK SUMMARY" \
  "$TASK22_LOG"

===== EXACT BENCHMARK SUMMARY =====
TASK 2-2 EXACT BENCHMARK SUMMARY
======================================================
Measured runs = 5
Mean seconds  = 1.242454
Stddev seconds = 0.126159
======================================================

======================================================
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 21: Benchmark Summary: Execution Times and Standard Deviations Across 5 Runs

5.5.2 Accuracy and Threshold Divergence

Figure 22 provides the divergence breakdown across all 16,486 groups:

Table 11: Approximate vs. Exact Discrepancy Statistics

Evaluation Metric	P90 Percentile	P80 Percentile
Groups with Threshold Difference	7,168 / 16,486 (43.5%)	6,053 / 16,486 (36.7%)
Groups with Qualifying Set Difference	305 / 16,486 (1.9%)	1,002 / 16,486 (6.1%)
Groups with Final Stddev Difference	125 / 16,486 (0.8%)	403 / 16,486 (2.4%)

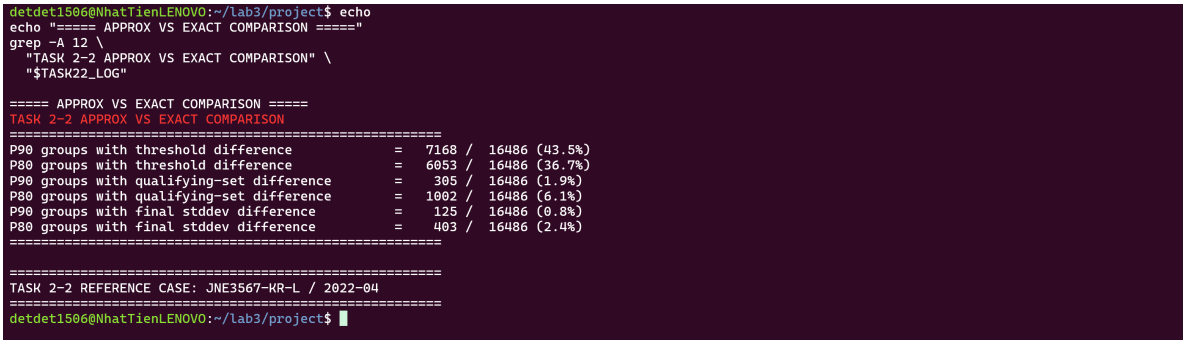

Figure 22: Divergence Summary: Threshold, Qualifying Set, and Stddev Differences

Although numerical threshold divergence occurs in 43.5% of cohorts for P90, qualifying order sets differ in only 1.9% of cohorts, altering final standard deviations in merely 0.8% of cases. Because promotion count is an integer attribute, threshold differences only affect order inclusion when orders exist strictly between the approximate and exact cutoffs.

5.5.3 Detailed Examination of the Instructor Reference Case

Figure 23 highlights the reference cohort: `SKU = JNE3567-KR-L`, `Month = 2022-04`, containing $N = 120$ orders.

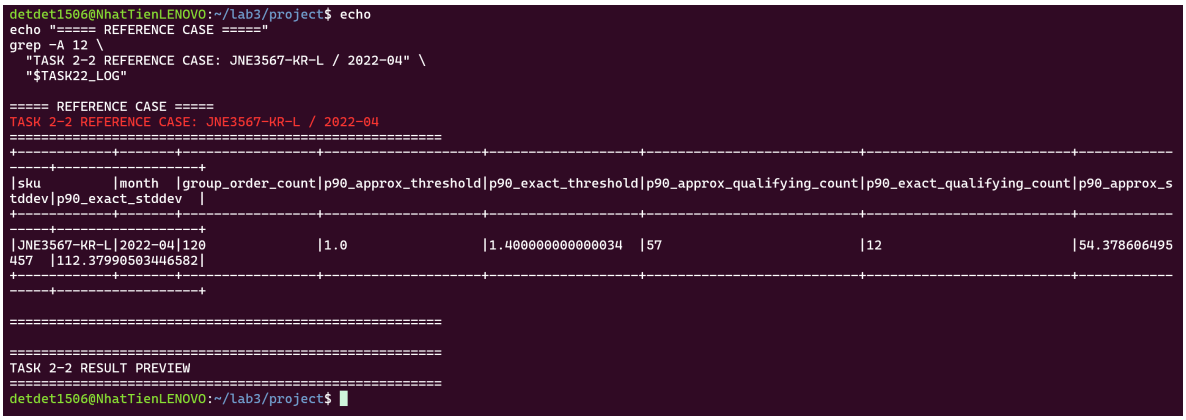

Figure 23: Reference Case Inspection for JNE3567-KR-L (2022-04)

- **Position Calculation:** $\text{pos} = (120 - 1) \times 0.90 = 107.1$.
- **Threshold Values:** APPROX yields **1.0**, whereas EXACT yields **1.4**.
- **Impact on Selection:** Within this cohort, 45 orders have a promotion count of exactly 1. The APPROX threshold (≥ 1.0) retains these 45 orders, yielding 57 qualifying orders with a standard deviation of **54.38**. The EXACT threshold (≥ 1.4) rejects these 45 orders, retaining only 12 orders with a standard deviation of **112.38**.

5.6 Partitioning Strategy and Large Group Discussion

The lab specification requires an evaluation of partitioning strategies if any cohort exceeds 1,000 orders:

1. **Empirical Cohort Volume:** As validated in Section 5.4.4, the maximum cohort size is 426 orders, and zero cohorts exceed 1,000 orders.
2. **Utility of Manual Repartitioning:** Manual repartitioning (e.g., `repartitionByRange`) is not recommended. Introducing explicit shuffle boundaries incurs unnecessary network serialization that outweighs any load-balancing benefit for datasets of this scale.
3. **Relation to Default Partition Size (128 MB):** An order record occupies approximately 50 to 80 bytes. A hypothetical cohort of 1,000 orders requires under 100 KB of memory—far below Spark's default 128 MB partition threshold. Data skew is completely negligible, allowing in-memory operations to run without risk of memory spills.

5.7 Final Parquet Artifact and Read-Back Verification

The final combined dataset is coalesced into a single partition and exported to the normal filesystem:

`~/lab3/results/Task_2-2.parquet`

5.7.1 Artifact Generation and Normal Filesystem Renaming

Spark writes partitioned outputs into a target directory containing metadata flags and chunk files (`part-*.parquet`). As demonstrated in Figure 24, the execution pipeline isolates exactly one Parquet part file. The surrounding workflow executes a direct filesystem rename to satisfy the required single-file naming convention:

Figure 25 demonstrates the execution of the relocation script, verifying that the resulting normal-filesystem artifact `Task_2-2.parquet` has a file size of **337 KB** and is verified by the OS utility as valid `Apache Parquet` binary data.

```
detdet1506@NhatTienLENOVO:~/lab3/project$ echo
echo "===== TEMP OUTPUT ====="
find "$TASK22_TMP" \
  -maxdepth 1 \
  -type f \
  -printf '%f\n'

part_count=$(find "$TASK22_TMP" \
  -maxdepth 1 \
  -type f \
  -name 'part-*.parquet' \
  | wc -l)

echo "Parquet part-file count = $part_count"

if [ "$part_count" -ne 1 ]; then
  echo "ERROR: Expected exactly one part-*.parquet file."
fi

===== TEMP OUTPUT =====
._SUCCESS.crc
_SUCCESS
.part-00000-6c4ec6e1-41ed-4bb7-8461-26ecbbb9c82b-c000.snappy.parquet.crc
part-00000-6c4ec6e1-41ed-4bb7-8461-26ecbbb9c82b-c000.snappy.parquet
Parquet part-file count = 1
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 24: Single Parquet Part-File Output Verification in Temporary Directory

```
detdet1506@NhatTienLENOVO:~/lab3/project$ task22_part=$(find "$TASK22_TMP" \
  -maxdepth 1 \
  -type f \
  -name 'part-*.parquet' \
  | head -n 1)

if [ -z "$task22_part" ]; then
  echo "ERROR: Could not find Task 2-2 Parquet part file."
else
  mv "$task22_part" "$TASK22_FILE"
  rm -rf "$TASK22_TMP"
  echo "OK: Created $TASK22_FILE"
fi

ls -lh "$TASK22_FILE"
file "$TASK22_FILE"
OK: Created /home/detdet1506/lab3/results/Task_2-2.parquet
-rw-r--r-- 1 detdet1506 detdet1506 337K Sep  4 12:29 /home/detdet1506/lab3/results/Task_2-2.parquet
/home/detdet1506/lab3/results/Task_2-2.parquet: Apache Parquet
detdet1506@NhatTienLENOVO:~/lab3/project$
```

Figure 25: Normal Filesystem Extraction, File Sizing (337K), and Parquet Validation

5.7.2 Independent Spark Read-Back Verification

Rather than relying solely on driver runtime logs, the produced Parquet file was subjected to independent assertions executed within a headless `spark-shell` environment:

- **Row Count Completeness:** Evaluates count $== 16,486$ matching the exact reference number of $(\text{SKU}, \text{Month})$ groups.
- **Primary Key Uniqueness:** Validates that grouping on `(sku, month)` yields zero duplicate keys across the entire table.
- **Value Bounds and Null Checking:** Verifies that all standard deviation outputs are strictly non-null and ≥ 0.0, and qualifying counts remain strictly within $[0, N]$.
- **Metric Invariant Replication:** Verifies that divergence rates across all groups match the slide reference tolerances (43.5%, 36.7%, 1.9%, 6.1%, 0.8%, and 2.4%).

As shown in Figure 26, all assertions passed unconditionally, confirming the mathematical correctness of both the built-in approximate quantile and self-implemented exact percentile pipelines.

Figure 26: Confirmation of All Task 2-2 Read-Back Assertions Passed in Spark Shell

References